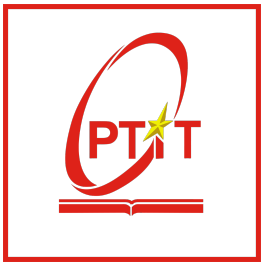
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO ĐỌC VÀ PHÂN TÍCH PAPER

MÔN HỌC: PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

MULTI-AGENT COLLABORATION VÀ REINFORCEMENT LEARNING

Giảng viên hướng dẫn	:	PGS.TS. Trần Đình Quế
Lớp	:	D23CTPM01
Nhóm học phần	:	CT01
Sinh viên thực hiện	:	Nguyễn Nam Hải
Mã sinh viên	:	B23DCCN277

Hà Nội – Năm 2026

MỤC LỤC

Danh mục thuật ngữ **iv**

1 Mở đầu: Hai paper trong một khung nghiên cứu **1**
1.1 Cấu trúc và quy ước bằng chứng 1

2 Paper 1: Cơ chế cộng tác đa tác tử dựa trên LLM **2**
2.1 Hồ sơ tài liệu, phạm vi và cách tiếp cận 2
2.2 Vấn đề nghiên cứu và động lực 2
2.3 Đóng góp và vị trí của survey 3
2.4 Nền tảng: MAS, LLM và collaborative AI 3
2.5 Mô hình hóa tác tử và hệ cộng tác 4
2.6 Cộng tác sớm, giữa và muộn 4
2.7 Ba loại quan hệ cộng tác 5
2.8 Ba chiến lược: rule-based, role-based, model-based 6
2.9 Cấu trúc giao tiếp và chi phí 7
2.10 Điều phối tĩnh và động 8
2.11 Bỏ phiếu, tranh luận và kiểm chứng 8
2.12 Ứng dụng được survey phân tích 9
2.13 Vấn đề mở và đánh giá phê bình 9
2.14 Thiết kế nghiên cứu tiếp nối do báo cáo đề xuất 10
2.15 Kết luận riêng cho Paper 1 11

3 Paper 2: Học tăng cường và ứng dụng trong các lĩnh vực **12**
3.1 Hồ sơ bài báo và mục đích đọc 12
3.2 RL khác học có giám sát và học không giám sát 12
3.3 Mô hình MDP và giả định Markov 12
3.4 Những thành phần cơ bản của môi trường 13
3.5 Return, hệ số chiết khấu và tiêu chí kinh doanh 13
3.6 Chính sách, kế hoạch và hàm giá trị 14
3.7 Phương trình Bellman 14
3.8 Dynamic programming: biết mô hình để lập kế hoạch 15
3.9 Monte Carlo: học từ return hoàn chỉnh 16
3.10 Temporal difference và ý nghĩa bootstrapping 16
3.11 SARSA và Q-learning 16
3.12 Epsilon-greedy và khám phá 17
3.13 Learning rate và điều kiện hội tụ 17
3.14 Actor–critic và policy gradient 18
3.15 Bayesian methods và bất định 18
3.16 Deep Q-Network và các thành phần ổn định 19
3.17 So sánh các họ phương pháp 19
3.18 Ứng dụng trong paper: đọc bằng cấu trúc bài toán 20

3.19 Các thách thức lớn và đọc phản biện . . . 21
3.20 Kiểm tra giả định trên chính môi trường assignment . . . 22
3.21 Phân tích lợi nhuận kỳ vọng và chính sách mốc . . . 23
3.22 Cầu nối với MAS và phương pháp nghiên cứu chung . . . 23
3.23 Kết luận riêng cho Paper 2 . . . 24

4 Chuyên đề toán học và kỹ thuật bổ sung 25
4.1 1. Từ collaboration channel đến hợp đồng có thể kiểm . . . 25
4.2 2. State machine cho workflow thay vì vòng chat không giới hạn . . . 25
4.3 3. Complexity của giao tiếp và đường găng . . . 26
4.4 4. Mô hình lỗi dây chuyền và giá trị kiểm chứng . . . 26
4.5 5. Đa số và tương quan sai số . . . 26
4.6 6. Confusion matrix, macro-F1 và selective prediction . . . 27
4.7 7. Vì sao chia theo nhóm dữ liệu quan trọng . . . 27
4.8 8. Chứng minh observation baseline chưa đủ Markov . . . 27
4.9 9. Hữu hạn chân trời và terminal aliasing . . . 28
4.10 10. Quy hoạch động lùi: một ví dụ hai bước . . . 28
4.11 11. Giá trị kỳ vọng của policy bằng forward occupancy . . . 29
4.12 12. Tách lỗi học khỏi nhiễu evaluation bằng exact values . . . 29
4.13 13. Kỳ vọng và phương sai profit ở một bước . . . 29
4.14 14. Confidence interval và đơn vị mẫu . . . 30
4.15 15. Robustness: zero-shot shift khác retraining . . . 30
4.16 16. Đối thủ cố định, self-play và cân bằng . . . 30
4.17 17. Từ thuật toán sang thiết kế có thể tái lập . . . 31
4.18 18. Phạm vi đóng góp của bộ báo cáo . . . 31

5 Tổng hợp, phản biện và thiết kế nối hai hướng nghiên cứu 32
5.1 Bảng đối chiếu mục tiêu và loại bằng chứng . . . 32
5.2 Hai cấp độ thích nghi không nên gộp . . . 32
5.3 Thiết kế tham chiếu: trợ lý kinh doanh có tầng quyết định và tầng kiểm chứng 32
5.4 Thiết kế thí nghiệm để tách lợi ích chia vai khỏi tăng tính toán . . . 33
5.5 Đề xuất kiểm tra độ bền nhiều tầng . . . 33
5.6 Những luận điểm cần tránh sau khi đọc hai paper . . . 33

6 Bằng chứng minh họa liên hệ hai paper 34

7 Phụ lục đọc paper: bản đồ nội dung và câu hỏi kiểm tra 35
7.1 Đối chiếu Paper 1 với tài liệu gốc . . . 35
7.2 Đối chiếu Paper 2 với tài liệu gốc . . . 36
7.3 Câu hỏi phản biện và lời giải định hướng . . . 37

8 Kết luận chung của báo cáo paper 37

Tài liệu tham khảo 38

DANH MỤC HÌNH VẼ

1 Minh họa phân biệt chất lượng trong phân phối và đánh giá ngoài miền; đây là dữ liệu tự thực thi. 34
2 Độ nhạy learning rate trong mô hình bán hàng toy; vùng tô là một SD qua năm seed. 35
3 Đánh giá policy đóng băng và mốc analytic không giảm giá. 35

DANH MỤC THUẬT NGỮ

MAS	Multi-Agent System – Hệ đa tác tử
LLM	Large Language Model – Mô hình ngôn ngữ lớn
RL / MARL	Reinforcement Learning / Multi-Agent RL
MDP / POMDP	Quá trình quyết định Markov / quan sát không đầy đủ
TD / Q	Sai phân thời gian / Giá trị hành động
DP	Dynamic Programming – Quy hoạch động
OOD	Out-of-distribution – Ngoài phân phối/ngoài miền
PEAS	Performance, Environment, Actuators, Sensors
DAG	Directed Acyclic Graph – Đồ thị có hướng không chu trình
SD / CI	Độ lệch chuẩn mẫu / Khoảng tin cậy
P50 / P95	Phân vị 50 / 95 của thời gian phản hồi
TTL	Time to Live – Giới hạn độ mới của dữ liệu

Quy ước đọc báo cáo

Dữ liệu thực hành là tổng hợp hoặc mô phỏng. Số đo được lấy từ mã đã chạy và lưu kèm; phép tính lý thuyết và đề xuất chưa thực hiện được gọi rõ. Thông tin bìa, logo và phong cách trình bày kế thừa báo cáo trước. Ngày hoàn thiện: 28/10/2026.

1 Mở đầu: Hai paper trong một khung nghiên cứu

Báo cáo đọc và phân tích đầy đủ hai tài liệu giảng viên cung cấp: survey về cơ chế cộng tác nhiều LLM agent và tổng quan học tăng cường trong các lĩnh vực. Phần thứ nhất trả lời cách tổ chức các thành viên, thông tin và quyết định; phần thứ hai trả lời cách policy học từ tương tác và reward. Cả hai được liên hệ với tutorial và Laboratory Experiment 02 đã thực thi trong cùng bộ bài.

Mục tiêu của báo cáo là giải thích đủ sâu để người đọc có thể chuyển từ khái niệm sang thiết kế, đồng thời nêu điều kiện và giới hạn của từng kết luận. Nội dung không dịch nguyên văn hai paper; phần diễn giải, phép tính bổ sung và thiết kế thực nghiệm do người viết phát triển được trình bày tách khỏi mô tả đóng góp của tác giả.

1.1 Cấu trúc và quy ước bằng chứng

Paper 1 được đọc theo background, mô hình tác tử, collaboration channels, loại quan hệ, chiến lược, topology, orchestration, ứng dụng và vấn đề mở. Paper 2 được đọc theo MDP, Bellman, các họ thuật toán, ứng dụng, thách thức và giả định. Phần tổng hợp cuối đối chiếu hai hướng, xây thiết kế mẫu và đề xuất protocol kiểm chứng.

Trong toàn bộ báo cáo, "kết quả paper" chỉ dùng cho những gì tài liệu gốc thực sự trình bày; "kết quả mô phỏng" chỉ dùng cho dữ liệu do mã đi kèm sinh; "đề xuất" chỉ phương án chưa được kiểm nghiệm đầy đủ. Không dùng demo cục bộ như bằng chứng tái lập benchmark LLM, không dùng dữ liệu tổng hợp như bằng chứng sẵn sàng triển khai thực tế.

2 Paper 1: Cơ chế cộng tác đa tác tử dựa trên LLM

2.1 Hồ sơ tài liệu, phạm vi và cách tiếp cận

Tài liệu được phân tích: Khanh-Tung Tran, Dung Dao, Minh-Duong Nguyen, Quoc-Viet Pham, Barry O'Sullivan và Hoang D. Nguyen, *Multi-Agent Collaboration Mechanisms: A Survey of LLMs*, arXiv:2501.06322v1, công bố ngày 10/01/2025, 35 trang. Báo cáo bám bản PDF giảng viên cung cấp, không thay bản này bằng một phiên bản khác xuất hiện sau đó. Các nhận định về nội dung paper được đối chiếu trực tiếp với Sections 1–7 và các bảng, hình trong bản PDF. [3]

Đây là một bài **survey về cơ chế phối hợp**, không phải paper đề xuất một thuật toán tối ưu duy nhất và đánh giá nó bằng một thí nghiệm thống nhất. Vì vậy, câu hỏi đọc hiểu thích hợp là: tác giả tổ chức lĩnh vực như thế nào, khung phân loại có giúp phân biệt thiết kế không, và các luận điểm tổng quát được hỗ trợ đến đâu? Việc chỉ kể tên AutoGen, MetaGPT hoặc các agent nổi tiếng sẽ bỏ qua đóng góp trọng tâm của bài.

Báo cáo triển khai ba lớp: diễn giải nội dung gốc; phân tích kỹ thuật và phản biện của người viết; liên hệ với mã tutorial và assignment đã chạy. Những phương trình chi phí, thiết kế kiểm chứng và ví dụ thương mại điện tử do báo cáo phát triển sẽ được gọi rõ là phân tích hoặc đề xuất. Không có thí nghiệm gọi LLM trong bộ mã đi kèm; do đó kết quả mô phỏng chỉ minh họa cơ chế phối hợp, không tái lập benchmark của các hệ LLM được paper tổng thuật.

2.2 Vấn đề nghiên cứu và động lực

Một LLM có thể tạo văn bản, viết mã và giải một số bài toán suy luận, nhưng vẫn có giới hạn về thông tin có sẵn, độ dài ngữ cảnh, độ tin cậy của hành động và khả năng tự kiểm tra. Tác giả đặt vấn đề chuyển từ cải thiện một mô hình đơn lẻ sang tổ chức nhiều tác tử có thể chia sẻ công việc. Ý tưởng hấp dẫn ở chỗ mỗi thành viên phụ trách một phần nhỏ hơn, có công cụ và góc nhìn riêng, rồi một cơ chế phối hợp tạo ra lời giải chung.

Tuy nhiên, phân chia công việc phát sinh một bài toán mới: làm sao kết quả của các thành viên tương thích? Một agent truy hồi có thể lấy đúng đoạn dữ liệu nhưng agent tổng hợp diễn giải sai; agent lập kế hoạch có thể đề xuất bước ngoài quyền của agent thực thi; hai agent có thể cùng sửa một tài nguyên. Khi ấy năng lực của hệ thống bị giới hạn bởi chất lượng quan hệ giữa các thành viên. Đó là lý do paper đặt **collaboration channels** ở trung tâm phân tích.

Một ví dụ xuyên suốt của báo cáo là trợ lý mua hàng. Agent danh mục truy hồi sản phẩm, agent đề xuất xếp hạng, agent kho kiểm tra số lượng và agent chính sách kiểm tra ngân sách. Nếu chỉ cộng bốn văn bản trả lời lại, hệ thống có thể khuyến nghị một sản phẩm hết hàng hoặc vượt ngân sách. Giá trị của phối hợp nằm ở điều kiện chuyển giao: ID sản phẩm phải tồn tại, giá phải lấy từ danh mục có thẩm quyền, thông tin kho phải còn mới và kết quả cuối phải được kiểm tra.

Phân tích của người viết: việc tăng số agent làm tăng số cơ hội trao đổi thông tin nhưng

cũng tăng số điểm có thể hỏng. Một mô hình đơn với công cụ tốt có thể hoàn thành bài đơn giản nhanh hơn một nhóm nhiều agent. Bởi vậy, lý do xây MAS cần gắn với yêu cầu phân quyền, chuyên môn, dữ liệu phân tán hoặc quyết định tương tác, rồi kiểm tra lợi ích bằng baseline tương xứng. Đây là một luận điểm kỹ thuật cần mang theo khi đọc các lời hứa về trí tuệ tập thể.

2.3 Đóng góp và vị trí của survey

Trong Section 1.2, tác giả nêu bốn hướng đóng góp: khảo sát cơ chế phối hợp giữa các LLM agent; đưa ra một framework tổng hợp; xem xét ứng dụng; và rút ra bài học cùng vấn đề mở. Bảng so sánh các survey ở phần đầu bài nhằm cho thấy nghiên cứu này tập trung vào phương diện *cách cộng tác diễn ra*, thay vì chỉ liệt kê ứng dụng hoặc mô tả vòng đời agent.

Đóng góp đầu tiên là một **ngôn ngữ phân tích chung**. Cùng dùng LLM nhưng một hệ có thể phối hợp qua bỏ phiếu, một hệ qua chia vai và một hệ qua mô hình dự đoán đối tác. Nếu gọi chung cả ba là "multi-agent reasoning", người đọc sẽ khó biết yếu tố nào tạo ra hiệu quả. Framework tách loại quan hệ, chiến lược, cấu trúc và điều phối, nhờ đó cho phép mô tả một thiết kế theo nhiều chiều độc lập.

Đóng góp thứ hai là nối các hướng nghiên cứu thường nằm riêng: MAS cổ điển, hệ nhiều LLM, cộng tác người–máy, chia sẻ mô hình và mô phỏng xã hội. Phạm vi rộng giúp nhận ra những điểm chung về thông tin, mục tiêu và điều phối. Đổi lại, phạm vi này khiến một vài ví dụ khác nhau đáng kể về cơ chế: một khách hàng federated learning và một tác tử hội thoại không có cùng chu trình hành động hay độ tự chủ. Khi áp dụng framework, cần ghi rõ cấp độ trừu tượng đang xét.

Về phương pháp, bài tự mô tả việc phân tích các nghiên cứu và tổng hợp đặc trưng. Báo cáo không tìm thấy một quy trình tìm kiếm, sàng lọc và đánh giá chất lượng theo chuẩn systematic review được mô tả đầy đủ như một sơ đồ PRISMA. Do đó nên đọc các bảng của bài như bản đồ có tổ chức của tài liệu được chọn, không xem đó là thống kê bao phủ hoàn toàn mọi công bố trong lĩnh vực. Đây là giới hạn phương pháp, không làm mất giá trị của taxonomy.

2.4 Nền tảng: MAS, LLM và collaborative AI

Section 2.1 mô tả MAS bằng bốn nhóm thành phần: tác tử, môi trường, tương tác và tổ chức. Tác tử có vai trò, năng lực và chính sách hành vi; môi trường cung cấp điều kiện để quan sát và hành động; tương tác gồm trao đổi thông tin, phối hợp hoặc thương lượng; tổ chức xác định quan hệ kiểm soát. Một hệ nhiều mô hình chỉ trở thành MAS theo nghĩa hữu ích khi có các thành viên và quan hệ tương tác đủ rõ để phân tích.

Section 2.2 đặt LLM vào vai trò bộ xử lý trung tâm của tác tử. Mô hình có thể tiếp nhận chỉ dẫn, suy luận trên văn bản và tạo lời gọi công cụ. Nhưng "LLM là bộ não" chỉ là phép diễn giải kiến trúc. Bộ nhớ, quyền truy cập, giao thức thông điệp, vòng lặp công cụ và tiêu chí dừng vẫn phải được thiết kế. Khả năng sinh một câu "đã kiểm tra kho" không đồng nghĩa với việc một truy vấn kho hợp lệ đã được thực hiện.

Section 2.3 xem collaborative AI như hướng phối hợp giữa các hệ AI hoặc giữa AI và con người. Hợp tác là một trường hợp; tranh luận, cạnh tranh và vừa hợp tác vừa cạnh tranh cũng thuộc phổ tương tác mà paper xem xét. Từ đó cần phân biệt mục tiêu ở hai cấp: hai thành viên có thể bất đồng ở cấp lập luận nhưng vẫn cùng phục vụ chất lượng đáp án cuối của hệ thống.

Ví dụ phản biện: một người viết và một người kiểm chứng có thể dùng hai system prompt khác nhau nhưng cùng một mô hình, cùng dữ liệu và cùng thiên kiến. Chia vai làm thay đổi quá trình xử lý, chưa chứng minh hai nguồn sai số độc lập. Muốn đánh giá tính bổ sung, cần đo tỷ lệ lỗi chung, độ đa dạng của bằng chứng và khả năng phát hiện sai của người kiểm chứng, thay vì đếm số persona.

2.5 Mô hình hóa tác tử và hệ cộng tác

Ở Section 3.1, tác giả biểu diễn một agent bằng bộ $a = \{m, o, e, x, y\}$: mô hình, mục tiêu, môi trường/ngữ cảnh, đầu vào và đầu ra. Thành phần mô hình được mô tả thêm bằng kiến trúc, bộ nhớ và adapter. Với LLM, đầu vào thường là chuỗi token, ngữ cảnh chịu giới hạn cửa sổ và đầu ra có thể là văn bản hoặc hành động được mã hóa. Ký hiệu này tổ chức khái niệm, không phải một bộ phương trình bảo đảm tính tối ưu.

Hệ cộng tác gồm tập tác tử $\mathcal{A}$, mục tiêu chung O, môi trường chia sẻ $\mathcal{E}$, tập kênh $\mathcal{C}$, đầu vào và đầu ra hệ thống. Có thể diễn giải ngắn gọn:

$$y_{\text{system}} = S(O, \mathcal{E}, x \mid \mathcal{A}, \mathcal{C}).$$

Điểm có ích là đầu ra phụ thuộc cả agent lẫn **kênh cộng tác**. Giữ nguyên các mô hình nhưng đổi ai nói trước, ai được thấy đầu ra nào và ai có quyền kết thúc có thể làm hành vi thay đổi. Điều này giải thích vì sao chỉ báo tên backbone chưa đủ để tái lập một thí nghiệm MAS.

Mỗi kênh cần mô tả: ai tham gia; mục tiêu hoặc nội dung trao đổi; loại quan hệ; cấu trúc kết nối; chiến lược đưa ra quyết định; điều kiện được tạo và kết thúc. Trong tutorial, lời gọi từ coordinator đến inventory là một kênh request–response hợp tác, tập trung, theo vai trò và có deadline. Đầu vào là danh sách ID; đầu ra là số lượng và tuổi dữ liệu. Mô tả này kiểm tra được bằng trace, khác với nhãn mơ hồ "agent kho hỗ trợ agent bán hàng".

Framework gốc giả định các agent có nền tảng giao tiếp chung. Phân tích của báo cáo: cùng dùng tiếng Anh hoặc JSON mới giải quyết một phần cú pháp. Hai bên còn phải thống nhất đơn vị tiền, nghĩa của "available", múi giờ, trạng thái lỗi, độ mới của dữ liệu và quyền hành động. Những chi tiết hợp đồng này quyết định liệu sự hiểu nhau có đúng ở mức vận hành hay chỉ đúng ở mức ngôn ngữ.

2.6 Cộng tác sớm, giữa và muộn

Paper mô tả cộng tác ở các giai đoạn khác nhau. Cộng tác sớm có thể chia sẻ dữ liệu, ngữ cảnh hoặc biểu diễn đầu vào; cộng tác giữa có thể trao đổi tham số, trọng số hoặc adapter; cộng tác muộn kết hợp đầu ra hoặc hành động. Ba mức cho thấy cộng tác không đồng nghĩa

với hội thoại bằng ngôn ngữ tự nhiên.

Chia sẻ ngữ cảnh giúp hai agent cùng thấy bằng chứng, nhưng có thể làm lan một nguồn sai chung. Chia sẻ mô hình giúp học từ nhiều nơi, nhưng kéo theo tương thích kiến trúc, quyền dữ liệu và bảo mật cập nhật. Kết hợp đầu ra đơn giản hơn về triển khai, nhưng không sửa được những lỗi đã bị mất khỏi các bản tóm tắt trung gian. Mỗi điểm kết nối làm thay đổi thứ thông tin còn có thể được kiểm chứng ở cuối quy trình.

Một thiết kế mua hàng có thể dùng cả ba mức: các agent đọc một catalog snapshot chung; mô hình intent được cập nhật theo dữ liệu tổng hợp; coordinator kết hợp các danh sách đề xuất đã qua kiểm tra. Tuy nhiên, bộ mã của báo cáo chỉ huấn luyện một classifier cục bộ và truyền dữ liệu có cấu trúc, không thực hiện federated learning hoặc trao đổi trọng số giữa nhiều tác tử. Việc liên hệ kiến trúc không phải tuyên bố đã triển khai mọi cấp độ trong survey.

2.7 Ba loại quan hệ cộng tác

2.7.1 Cooperation: mục tiêu được căn chỉnh

Trong hợp tác, các tác tử chia công việc để tăng chất lượng mục tiêu chung. Chuyên môn hóa giúp giảm phạm vi mỗi thành viên phải xử lý. Người tìm tài liệu có thể tập trung vào độ bao phủ; người kiểm chứng tập trung vào dẫn chứng; người biên tập tập trung vào tính mạch lạc. Các thành phần khác nhau không cần tối ưu một chỉ số cục bộ giống hệt nhau, nhưng kết quả phải phục vụ cùng tiêu chí nghiệm thu.

Rủi ro là mục tiêu cục bộ dễ lệch. Nếu agent đề xuất chỉ tối ưu điểm hấp dẫn, nó có thể chọn sản phẩm vượt ngân sách. Nếu agent tổng hợp chỉ tối ưu độ đầy đủ, nó có thể bổ sung thông tin không có trong nguồn. Vì vậy cần một hàm mục tiêu hoặc tập ràng buộc ở cấp hệ thống, cùng cơ chế kiểm tra đầu ra từng bước. “Cùng giúp người dùng” chưa phải hợp đồng phối hợp đủ chặt.

2.7.2 Competition: bất đồng có cấu trúc

Cạnh tranh xuất hiện khi thành viên tối ưu lợi ích riêng hoặc được giao bảo vệ các lập luận trái chiều. Tranh luận có thể làm lộ giả định và lỗi suy luận, đặc biệt khi mỗi bên phải viện dẫn bằng chứng. Tuy nhiên, một agent phản đối mạnh không nhất thiết đưa ra phản biện đúng. Cơ chế phán quyết cần ưu tiên tính kiểm chứng, thay vì văn phong tự tin hoặc số lượng phát biểu.

Ví dụ của người viết: agent A đề xuất chọn sản phẩm p1 vì hiệu năng, agent B phản biện rằng p1 vượt ngân sách. Nếu giá p1 được xác nhận từ catalog, đây là phản biện có giá trị quyết định. Nếu B chỉ nói “có thể đắt”, coordinator chưa có đủ cơ sở. Sự khác biệt giữa phản đối và kiểm chứng phải được phản ánh trong dữ liệu thông điệp: claim, evidence, constraint và verdict nên là các trường riêng.

2.7.3 Coopetition: hợp tác ở một cấp, cạnh tranh ở cấp khác

Trong coopetition, các bên có lợi ích vừa trùng vừa khác. Hai công ty có thể cùng dùng hạ tầng logistics nhưng cạnh tranh khách hàng; nhiều nhóm có thể phối hợp chia tài nguyên tính toán nhưng cạnh tranh thứ hạng. Thiết kế phải nêu rõ thứ gì được chia sẻ và thứ gì cần giữ riêng. Một mục tiêu chung không xóa bỏ xung đột lợi ích ở cấp tác tử.

Assignment bán hàng có hai lợi nhuận riêng và ảnh hưởng qua giảm giá. Nó là tương tác cạnh tranh về lợi nhuận, nhưng không tự động là trò chơi tổng bằng không: lượng mua của hai cửa hàng được lấy mẫu riêng, và không có ràng buộc tổng doanh số bằng một thị trường cố định. Gọi mọi biến động lợi nhuận là "một bên thắng thì bên kia thua" sẽ sai với mô hình đang dùng.

2.8 Ba chiến lược: rule-based, role-based, model-based

Chiến lược	Cơ chế chính	Thứ phải khai báo	Điểm dễ sai
Rule-based	Luật quyết định ai nói, nói khi nào, kết hợp ra sao	Điều kiện, ưu tiên, tiêu chí dừng	Luật thiếu trường hợp, xung đột thứ tự
Role-based	Phân công chuyên môn và trách nhiệm	Quyền, đầu vào, đầu ra, phạm vi	Chồng vai, bỏ sót nhiệm vụ, phụ thuộc dây chuyền
Model-based	Dự đoán môi trường, đối tác hoặc bất định để quyết định	Mô hình, quan sát, giả định, cách cập nhật	Dự đoán đối tác sai, chi phí lớn, khó kiểm chứng

2.8.1 Giao thức theo luật

Rule-based trong survey mô tả **giao thức tương tác**, chẳng hạn thứ tự tranh luận hoặc bỏ phiếu. Nó không đồng nhất với rule-based seller của assignment, nơi luật xác định hành động giảm giá. Một hệ có thể có agent học máy nhưng phối hợp theo luật cố định; ngược lại, một coordinator học cách phân việc có thể điều khiển các công cụ hoàn toàn xác định. Giữ riêng hai cấp tránh nhầm "có luật" với "không có học".

Ưu điểm của luật là truy vết được: mỗi quyết định có điều kiện và nhánh xử lý cụ thể. Nhược điểm là khi không gian tình huống lớn, số điều kiện tăng và tương tác giữa luật khó quản lý. Thiết kế tốt thường giữ các bất biến quan trọng bằng luật, rồi cho phép mô hình đề xuất phương án bên trong miền hợp lệ. Đây là cấu trúc hybrid phù hợp với tutorial.

2.8.2 Giao thức theo vai trò

Role-based đặt câu hỏi ai có chuyên môn hoặc tài nguyên thích hợp. MetaGPT là ví dụ paper dùng để minh họa quy trình có vai trò và SOP; AutoGen cung cấp cách tổ chức các agent có thể hội thoại, dùng công cụ hoặc tương tác với con người. Những ví dụ này cho thấy cấu trúc vai trò ảnh hưởng đến cách phân rã nhiệm vụ; chúng không tạo ra định lý

rằng mọi quy trình nhiều vai đều tốt hơn mọi baseline. [8, 7]

Một vai trò cần có khả năng bị kiểm tra và thay thế. Trong tutorial, CatalogAgent chỉ biết truy hồi ID; InventoryAgent chỉ cung cấp thông tin kho; PolicyAgent xác nhận nguồn gốc và ngân sách. Nếu thay ranker mà các hợp đồng không đổi, hệ còn chạy được. Đây là lợi ích thiết kế cụ thể của chia vai, ngay cả khi chất lượng tác vụ không tăng trên catalog nhỏ.

2.8.3 Giao thức dựa trên mô hình

Model-based trong paper có thể bao gồm suy luận xác suất và Theory of Mind để dự đoán mục tiêu hoặc hành vi của bên khác. Thuật ngữ này có phạm vi khác "model-based RL" trong paper thứ hai, nơi mô hình thường là chuyển trạng thái và phần thưởng. Cả hai cùng dùng mô hình để hỗ trợ quyết định nhưng đối tượng mô hình hóa và bảo đảm lý thuyết khác nhau.

Nếu agent bán hàng dự đoán xác suất đối thủ giảm giá ở bước kế tiếp, nó có thể tính lợi nhuận kỳ vọng của các lựa chọn. Nhưng nếu đối thủ cũng cập nhật chiến lược, dự đoán vừa học sẽ lỗi thời. Cần đo chất lượng mô hình đối tác trên dữ liệu tương lai và đánh giá chính sách dưới nhiều loại đối thủ. Một mô hình mô tả tốt lịch sử chưa chắc dẫn đến quyết định tốt trong tương tác chiến lược.

2.9 Cấu trúc giao tiếp và chi phí

2.9.1 Tập trung

Trong cấu trúc tập trung, coordinator nhận yêu cầu, phân việc, thu kết quả và quyết định đầu ra. Nó giúp đặt ràng buộc và logging ở một điểm. Tutorial chọn cấu trúc này để trình tự kiểm chứng rõ và dễ tái lập. Hạn chế là coordinator có thể thành nút nghẽn và điểm hỏng chung. Chia các worker độc lập không giải quyết hỏng của chính bộ điều phối.

Với n worker, một lượt request–response đến mọi worker cần xấp xỉ $2n$ thông điệp nếu bỏ qua retry. Chi phí tuyến tính này thuận tiện để dự toán. Tuy nhiên, mỗi request có thể chứa toàn bộ ngữ cảnh rất dài; số thông điệp nhỏ chưa chắc số token nhỏ. Vì vậy cần đo đồng thời số message, dung lượng payload và thời gian xử lý.

2.9.2 Phi tập trung và phân tán

Trong cấu trúc phi tập trung, quyết định được chia giữa các thành viên. Cấu trúc phân tán nói về vị trí thực thi; phi tập trung nói về quyền kiểm soát. Một hệ chạy trên nhiều máy nhưng mọi quyết định qua một manager vẫn có thể là tập trung về điều khiển. Đây là phân biệt kỹ thuật của báo cáo để làm rõ những khái niệm được đặt gần nhau trong survey.

Nếu mọi agent trao đổi với mọi agent khác, một vòng có tối đa $n(n-1)$ thông điệp có hướng. Với $n = 10$, đó là 90 thông điệp, chưa tính phản hồi và số vòng. Topology thưa, publish–subscribe hoặc giao tiếp theo sự kiện giúp giảm lưu lượng, nhưng cũng giới hạn thông tin mỗi thành viên nhìn thấy. Không có một topology tốt nhất độc lập với nhiệm vụ.

2.9.3 Phân cấp

Cấu trúc phân cấp đưa các nhóm agent dưới manager cấp trung rồi tổng hợp lên trên. Nó giúp chia việc và giảm bề rộng ở nút gốc, nhưng làm tăng số chặng truyền. Thông tin bị tóm tắt qua nhiều cấp có thể mất chi tiết cần kiểm chứng. Một cơ chế tốt cho phép truy ngược từ kết luận về bằng chứng gốc, thay vì chỉ chuyển bản tóm tắt của bản tóm tắt.

Một cây phân cấp còn đòi hỏi chính sách khi manager con hỏng: giao lại nhóm, trả kết quả một phần hay dừng toàn nhiệm vụ? Lựa chọn phụ thuộc tác vụ. Một bản gợi ý mua hàng có thể loại sản phẩm chưa kiểm chứng; một quy trình giao dịch tài chính không thể tự coi phần thiếu là thành công. Cấu trúc giao tiếp phải gắn với nghĩa của thất bại.

2.10 Điều phối tĩnh và động

Điều phối tĩnh thiết lập kênh và thứ tự trước khi nhận yêu cầu. Một pipeline catalog → ranker → policy → inventory là ví dụ. Ưu điểm là dễ kiểm tra số bước tối đa, quyền gọi công cụ và tình huống lỗi. Nó phù hợp khi cấu trúc tác vụ ổn định. Thêm một LLM vào từng bước không biến một quy trình đã cố định thành điều phối động.

Điều phối động cho phép tạo vai trò, chọn thành viên hoặc thay đồ thị thực hiện theo yêu cầu. Cách này có thể tiết kiệm khi chỉ gọi chuyên gia cần thiết, nhưng manager phải giải thêm bài toán lập kế hoạch. Một sơ đồ động sai có thể bỏ mất bước kiểm chứng bắt buộc. Do đó phần động nên chịu một lớp ràng buộc tĩnh: các bước tùy chọn có thể đổi, còn bất biến về nguồn gốc, quyền và xác nhận phải giữ.

Phân tích đồ thị của báo cáo: biểu diễn các bước bằng DAG $G = (V, E)$, cạnh thể hiện phụ thuộc dữ liệu. Tổng thời gian xấp xỉ đường găng cộng overhead điều phối; tổng chi phí bằng tổng công việc của mọi node được thực hiện. Song song hóa có thể giảm thời gian mà tăng tổng chi phí. Một đánh giá chỉ báo latency sẽ bỏ qua trade-off này.

2.11 Bỏ phiếu, tranh luận và kiểm chứng

Bỏ phiếu đa số đơn giản, nhưng chỉ có lợi khi lỗi giữa các thành viên có mức độc lập đủ lớn. Giả sử ba bộ trả lời độc lập, mỗi bộ đúng với xác suất p, xác suất đa số đúng là $3p^2(1-p)+p^3$. Với $p = 0.7$, kết quả là 0.784. Đây là ví dụ xác suất do báo cáo tính, không phải kết quả thực nghiệm của survey.

Nếu cả ba sao chép cùng một lời giải sai, tính độc lập không còn. Khi sai số tương quan hoàn toàn, bỏ phiếu không cải thiện xác suất đúng so với một thành viên. Vì vậy nên ưu tiên đa dạng bằng chứng, công cụ hoặc cách kiểm tra thay vì chỉ đổi tên vai. Đếm phiếu có thể làm tăng độ tự tin trong khi không tăng độ đúng.

Tranh luận nhiều vòng cho phép sửa câu trả lời theo phản biện. Nghiên cứu của Du và cộng sự là một ví dụ gốc về cách nhiều mô hình đề xuất rồi trao đổi qua nhiều vòng. Nhưng kết quả cải thiện trên một số tác vụ không phải bảo đảm cho mọi cấu hình agent. Khi tái lập cần công bố số agent, số vòng, prompt, cách lấy đáp án và tổng ngân sách suy luận. [9]

Kiểm chứng bằng công cụ có vị trí khác bỏ phiếu. Nếu yêu cầu là tính tổng, trình tính

có thể đưa ra phép kiểm xác định. Nếu yêu cầu là xác nhận còn hàng, chỉ dịch vụ kho có thẩm quyền mới trả lời được. Hai agent đồng ý rằng "còn hàng" không thay thế truy vấn kho. Bài học này được triển khai trực tiếp bằng trường `age_seconds` và kiểm tra freshness trong tutorial.

2.12 Ứng dụng được survey phân tích

2.12.1 Mạng 5G/B6G và Industry 5.0

Section 5.1 bàn về các hệ mạng, phân bổ tài nguyên, môi trường công nghiệp và phối hợp thông minh. Giá trị của MAS nằm ở quyết định phân tán, dữ liệu ở nhiều nơi và yêu cầu điều chỉnh theo trạng thái. Một tác tử cục bộ có thể phản ứng nhanh, trong khi cơ chế phối hợp tránh việc tối ưu cục bộ làm xấu chất lượng toàn mạng.

Thách thức triển khai là deadline, giới hạn tài nguyên và độ tin cậy. Một LLM tạo một phương án có vẻ hợp lý nhưng trả lời sau thời điểm cần hành động vẫn có thể không dùng được. Vì thế báo cáo đề xuất đánh giá tỷ lệ đáp ứng hạn, chi phí suy luận và vi phạm ràng buộc, bên cạnh chất lượng phương án. Phần này là cách chuyển chủ đề ứng dụng của survey thành tiêu chí kỹ thuật.

2.12.2 Hỏi đáp và sinh ngôn ngữ

Section 5.2 xem xét phân rã câu hỏi, sử dụng chuyên gia, phản biện và tổng hợp. Một câu hỏi nhiều bước có thể cần truy hồi từ nhiều nguồn rồi hợp nhất. Chia vai giúp quan sát các bước trung gian, nhưng cũng tạo thêm khả năng mất ngữ cảnh hoặc bóp méo bằng chứng. Cần lưu provenance từ mỗi claim đến đoạn nguồn.

Đánh giá chỉ bằng độ trôi chảy sẽ bỏ qua sự thật. Với hỏi đáp, nên tách độ đúng của đáp án, độ đúng của dẫn chứng, độ bao phủ và chi phí. Với viết mã, cần test hoặc đặc tả; với tổng hợp tài liệu, cần kiểm tra mâu thuẫn nguồn. LLM làm giám khảo có thể hỗ trợ nhưng bản thân giám khảo cũng cần đánh giá sai số.

2.12.3 Miền xã hội và văn hóa

Section 5.3 khảo sát mô phỏng cộng đồng, tương tác xã hội, nhân vật và hoạt động mang tính văn hóa. Các môi trường này thích hợp để nghiên cứu hành vi emergent vì nhiều thành viên có ký ức và quan hệ khác nhau. Tuy nhiên, một hành vi trông giống con người trong mô phỏng không đủ chứng minh mô hình phản ánh dân số thực.

Phản biện của người viết: kết quả mô phỏng chịu ảnh hưởng của prompt, dữ liệu tiền huấn luyện, giả định tác giả và cách thiết kế phần thưởng. Nếu dùng mô phỏng để hỗ trợ quyết định xã hội, cần đối chiếu với dữ liệu và chuyên gia bên ngoài. Gán quá nhiều thuộc tính tâm lý cho agent có thể khiến người đọc hiểu sai phạm vi của bằng chứng.

2.13 Vấn đề mở và đánh giá phê bình

2.13.1 Quản trị và quyết định tập thể

Section 6.1 đặt ra thống nhất quản trị, phân vai, phục hồi lỗi và ra quyết định chung. Một hệ đáng tin phải biết ai có quyền sửa mục tiêu, ai được gọi công cụ ghi và ai xác nhận hoàn tất. Khi manager và worker bất đồng, không thể chỉ thêm một vòng chat vô hạn; phải có cơ chế giải quyết hoặc trả quyền quyết định về người dùng.

Đề xuất của báo cáo là dùng ngân sách rõ cho vòng sửa: số lần thử, thời gian và chi phí tối đa; lưu lý do thay đổi; phân biệt "không tìm thấy lời giải" với "lỗi dịch vụ". Bộ điều phối cần có các trạng thái kết thúc thành công, kết quả một phần, thiếu bằng chứng, hết hạn và vi phạm hợp đồng. Cách mô hình hóa này biến một cuộc hội thoại thành quy trình có thể vận hành.

2.13.2 Benchmark và nguyên nhân cải thiện

Section 6.2 nhấn mạnh thiếu chuẩn đánh giá thống nhất. Một hệ nhiều agent có thể dùng nhiều token hơn baseline một agent, nên chênh lệch điểm chưa tách được tác dụng của phối hợp khỏi tác dụng của thêm tính toán. Baseline nên có phiên bản ngân sách tương đương, ví dụ một agent được dùng cùng số lượt công cụ hoặc cùng tổng token.

Ablation phải bỏ đúng thành phần đang muốn đo. Bỏ coordinator validation trong tutorial kiểm tra giá trị của validation; nó không chứng minh việc chia nhiều agent tạo ra cải thiện. So sánh B0 và B1 trên cùng bộ ca mới cho biết overhead của chia vai. So sánh B1 và B2 mới giúp xem classifier bổ sung có thay đổi khả năng định tuyến hay không.

2.13.3 An toàn, tấn công và độ tin cậy

Section 6.3 lưu ý lỗi và thông tin sai có thể lan qua các kênh. Một message từ agent nội bộ vẫn có thể chứa dữ liệu không đáng tin vì agent đó đọc nguồn ngoài. Cần tách dữ liệu khỏi chỉ dẫn, xác thực schema và giới hạn quyền công cụ. Trong tutorial, một ID lạ hoặc sản phẩm vượt ngân sách bị PolicyAgent bác bỏ trước khi có kết luận sẵn hàng.

Không nên suy diễn rằng thêm nhiều agent luôn tăng an toàn. Mỗi agent thêm một bề mặt lỗi và một chuỗi phụ thuộc. Một thiết kế phòng vệ có ích khi bước kiểm chứng có thẩm quyền và thực sự chặn hành động sai. Bản ablation B3 minh họa hệ có nhiều thành viên nhưng vẫn đưa ra thông tin sai khi bỏ kiểm tra nguồn gốc và tuổi dữ liệu.

2.14 Thiết kế nghiên cứu tiếp nối do báo cáo đề xuất

Câu hỏi nghiên cứu: khi catalog nhỏ và các ràng buộc xác định, chia vai có cải thiện tỷ lệ hoàn thành so với pipeline đơn không? Câu hỏi phụ: verification đóng góp bao nhiêu, classifier intent đóng góp bao nhiêu và độ trễ tăng bao nhiêu? Đây là các câu hỏi đủ hẹp để có đối chứng bằng mã cục bộ, thay vì khẳng định chung về mọi MAS.

Thiết kế gồm bốn kiến trúc B0–B3, cùng catalog và tập yêu cầu. Dữ liệu lỗi được tạo có chủ đích: timeout, sai envelope, dữ liệu kho cũ và output giả từ thành phần. Ground truth là trạng thái và danh sách ID hợp lệ; thành công được tính cả các ca từ chối đúng. Điều này

khác với chỉ đếm số lần hệ trả được danh sách sản phẩm.

Để mở rộng thành nghiên cứu LLM, giữ nguyên hợp đồng công cụ và bộ kiểm chứng rồi thay bộ đề xuất bằng LLM. Cần công bố model version, decoding, prompt, ngân sách, số vòng, dữ liệu và trace. Chạy nhiều seed hoặc nhiều lần lấy mẫu; phân tích theo loại lỗi; báo cáo cả phân phối chi phí. Bộ mã hiện tại là nền tảng kiểm thử kiến trúc, chưa phải thí nghiệm này.

2.15 Kết luận riêng cho Paper 1

Giá trị lớn nhất của paper là đổi trọng tâm từ số lượng agent sang cấu trúc quan hệ giữa chúng. Năm chiều actors, types, strategies, structures và coordination cho phép mô tả khá nhiều hệ trong một khung chung. Khi chuyển sang phần mềm, khung đó cần được cụ thể hóa bằng schema, quyền, nguồn dữ liệu, deadline và phép kiểm chứng.

Giới hạn cần giữ rõ là phạm vi survey rộng, các kết quả được tổng thuật đến từ những bài có cấu hình và benchmark khác nhau, và nhiều kỳ vọng về trí tuệ tập thể còn là hướng nghiên cứu. Báo cáo vì thế sử dụng paper để tổ chức câu hỏi thiết kế, rồi dùng các thực nghiệm cục bộ để kiểm tra những nhận định hẹp mà bộ mã thực sự đo được.

3 Paper 2: Học tăng cường và ứng dụng trong các lĩnh vực

3.1 Hồ sơ bài báo và mục đích đọc

Bài *A Gentle Introduction to Reinforcement Learning and Its Application in Different Fields* do Muddasar Naeem, Syed Tahir Hussain Rizvi và Antonio Coronato thực hiện, đăng trên IEEE Access, tập 8, năm 2020, trang 209320–209344, DOI 10.1109/ACCESS.2020.3038605. Bản PDF được cung cấp có 25 trang, gồm nền tảng MDP, nguyên lý RL, các nhóm thuật toán, ứng dụng và thảo luận. [4]

Mục đích của bài là cung cấp một lộ trình nhập môn tương đối rộng, giúp người đọc hiểu những khái niệm trước khi bước vào thuật toán chuyên sâu. Đóng góp phù hợp nhất để đánh giá là cách hệ thống hóa và nối kiến thức với ứng dụng. Bài không cung cấp một benchmark thống nhất chạy mọi thuật toán trên mọi lĩnh vực, nên không thể suy ra một bảng xếp hạng phổ quát từ số lượng ví dụ được nhắc đến.

Báo cáo đọc paper theo chuỗi: mô hình quyết định → đại lượng cần học → cách ước lượng → điều kiện áp dụng → kiểm tra bằng assignment. Các công thức được viết lại với ký hiệu nhất quán và kỳ vọng rõ ràng. Đây là phần diễn giải kỹ thuật của báo cáo, giúp tránh những giản lược ký hiệu trong tài liệu nhập môn. Ví dụ số và kết quả mô phỏng được phân biệt rõ với nội dung nguyên bản.

3.2 RL khác học có giám sát và học không giám sát

Trong học có giám sát, một quan sát đi cùng nhãn hoặc giá trị mục tiêu. Mô hình học quan hệ giữa đặc trưng và nhãn trên tập dữ liệu cho trước. Trong học không giám sát, mục tiêu thường là tìm cấu trúc, biểu diễn hoặc nhóm trong dữ liệu không có nhãn tác vụ. Trong RL, tác tử chọn hành động rồi nhận phản hồi; hành động ảnh hưởng dữ liệu quan sát tiếp theo, vì vậy quá trình thu thập dữ liệu gắn với chính sách hiện tại.

Phần thưởng khác nhãn đáp án. Khi cửa hàng giảm giá và nhận lợi nhuận 30, môi trường không nói hành động còn lại sẽ cho lợi nhuận bao nhiêu trong cùng tình huống. Tác tử phải học thông qua nhiều lần tương tác, đối mặt với nhiễu và đánh đổi giữa thử mới với khai thác điều đã biết. Nếu hậu quả xuất hiện chậm, còn phải xác định hành động nào trong chuỗi đóng góp cho kết quả cuối.

Tutorial intent sử dụng **supervised learning**: mỗi câu có nhãn ecommerce, tourism hoặc healthcare administration và mạng tối ưu cross entropy. Assignment sử dụng **reinforcement learning**: agent chọn hành động giảm giá và học từ lợi nhuận ngẫu nhiên. Hai phần đều có quá trình training nhưng không nên gọi classifier là RL. Tương tự, một hệ có nhiều agent không tự động là MARL nếu các thành viên không cập nhật chính sách từ tương tác.

3.3 Mô hình MDP và giả định Markov

Một MDP thường được biểu diễn bằng $(\mathcal{S}, \mathcal{A}, P, R, \gamma, \rho_0)$: không gian trạng thái, hành động, xác suất chuyển, phần thưởng kỳ vọng, hệ số chiết khấu và phân phối khởi đầu. Với bài hữu

hạn chân trời, cần nêu thêm H hoặc dùng trạng thái chứa thời gian. MDP là mô hình bài toán; RL là một họ phương pháp học quyết định, thường khi không biết đầy đủ P và R.

Tính Markov có thể viết:

$$P(S_{t+1}, R_{t+1} \mid S_0, A_0, \ldots, S_t, A_t) = P(S_{t+1}, R_{t+1} \mid S_t, A_t).$$

Điều kiện không nói quá khứ không quan trọng trong thế giới thực, mà nói trạng thái đã tóm tắt đủ thông tin quá khứ liên quan đến tương lai. Nếu trạng thái thiếu tồn kho, đơn hàng đang xử lý hoặc hành động trước mà đối thủ dùng để phản ứng, điều kiện có thể không còn. Một chuỗi quan sát có dạng vector không mặc nhiên là trạng thái Markov.

Cần tách tính Markov và tính dừng. Markov liên quan đến thông tin cần điều kiện hóa; tính dừng liên quan đến việc cùng một trạng thái và hành động có cùng quy luật qua thời gian. Một quá trình có thể Markov nhưng quy luật phụ thuộc thời điểm. Có thể bổ sung thời gian hoặc chế độ vào trạng thái để mô hình hóa sự phụ thuộc đó, nhưng khi không quan sát chế độ thì bài toán vẫn có bất định.

3.4 Những thành phần cơ bản của môi trường

Trạng thái phải chứa thông tin cần cho quyết định, không nhất thiết là mọi dữ liệu có thể lưu. Thêm quá nhiều biến làm tăng độ phức tạp; bỏ biến quan trọng làm giá trị ước lượng bị trộn. Trong assignment, quan sát tham chiếu có hai bit nên chỉ có bốn trạng thái cục bộ. Khi kiểm tra giả định Markov, báo cáo chỉ ra cần thêm hành động trước của chính mình để mô tả phản ứng của rule opponent.

Hành động là lựa chọn thực sự được môi trường cho phép. Ở đây $a \in \{0, 1\}$, không giảm và giảm 20%. Nếu mở rộng sang nhiều mức giảm, cần kiểm tra biên lợi nhuận và tập hợp hành động hợp lệ. Một mạng xuất ra giá trị liên tục không tự giải quyết tính hợp lệ của giao dịch hoặc hạn mức kinh doanh.

Chuyển trạng thái mô tả điều xảy ra sau hành động. Nhu cầu trong mô phỏng được lấy độc lập với hành động, nhưng hành động hiện tại trở thành lịch sử quan sát ở bước sau. Vì rule opponent phản ứng theo lịch sử này, agent vẫn có ảnh hưởng đến tương lai dù không điều khiển nhu cầu. Chỉ nhìn nhu cầu độc lập rồi kết luận bài toán là bandit sẽ bỏ qua cơ chế phản hồi đối thủ.

Phần thưởng phản ánh mục tiêu đo được. Trong mô phỏng là lợi nhuận số học, không phải doanh thu hoặc số đơn. Việc chọn reward là lựa chọn mô hình hóa có hệ quả: tối ưu số lượng bán sẽ có động lực khác tối ưu lợi nhuận; thêm phạt tồn kho hay phạt biến động giá lại tạo chính sách khác. Cần xác định mục tiêu trước khi chọn thuật toán.

3.5 Return, hệ số chiết khấu và tiêu chí kinh doanh

Return hữu hạn bắt đầu từ bước t được viết:

$$G_t = \sum_{k=0}^{H-t-1} \gamma^k R_{t+k+1}.$$

Khi $\gamma = 0$, chỉ phần thưởng kế tiếp được xem xét. Khi $0 < \gamma < 1$, các phần thưởng xa hơn có trọng số nhỏ hơn và chuỗi vô hạn bị chặn nếu reward bị chặn. Khi $\gamma = 1$, nhiệm vụ hữu hạn vẫn hợp lệ; trong nhiệm vụ vô hạn phải có điều kiện bổ sung để tổng có nghĩa. Vì vậy không thể nói mọi lựa chọn $\gamma = 1$ đều sai hoặc đều an toàn.

Có một khác biệt quan trọng trong assignment: cập nhật Q dùng $\gamma = 0.9$, trong khi báo cáo hiệu năng yêu cầu lợi nhuận trung bình không chiết khấu qua 20 bước. Hai tiêu chí có liên quan nhưng không đồng nhất trong mọi môi trường. Nếu một hành động hy sinh lợi nhuận đầu kỳ để có lợi ích cuối kỳ, việc giảm trọng số tương lai có thể thay đổi lựa chọn tối ưu theo chỉ số kinh doanh.

Trong mô hình hiện tại, quy hoạch động xác nhận chính sách không giảm giá là best response với rule opponent ở các tham số được dùng. Do đó khác biệt tiêu chí không làm thay đổi mốc hành động này. Tuy nhiên, báo cáo vẫn nêu sự không đồng nhất vì nó quan trọng khi thêm tồn kho, ngân sách khuyến mãi hoặc khách hàng quay lại.

Ví dụ của báo cáo: chuỗi reward $(5, 5, 20)$ có tổng 30, nhưng return với $\gamma = 0.9$ là $5 + 0.9 \times 5 + 0.9^2 \times 20 = 25.7$. Một chính sách khác nhận $(12, 10, 6)$ có tổng 28 và return 25.86. Như vậy chính sách có tổng ít hơn vẫn có thể được ưu tiên dưới chiết khấu. Ví dụ này cho thấy phải công bố rõ tối ưu đại lượng nào.

3.6 Chính sách, kế hoạch và hàm giá trị

Chính sách xác định cách hành động theo trạng thái hoặc lịch sử. Chính sách xác định viết $a = \pi(s)$; chính sách ngẫu nhiên viết $\pi(a \mid s)$. Một kế hoạch là chuỗi hoặc đồ thị bước để thực hiện, còn chính sách là quy tắc phản ứng cho những tình huống có thể gặp. Trong môi trường ngẫu nhiên, một danh sách hành động cố định thường không đủ thay thế chính sách.

Hàm giá trị trạng thái và hành động được định nghĩa bằng kỳ vọng:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s], \qquad Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

Một reward lớn trong một lượt không bằng Q lớn. Q phản ánh lợi ích dài hạn kỳ vọng khi thực hiện hành động rồi tiếp tục theo chính sách. Mẫu sales có nhiễu, nên hai lần cùng trạng thái và hành động vẫn có thể cho reward khác nhau. Bảng Q giảm bớt nhiễu nhờ cập nhật qua nhiều quan sát, nhưng cách giảm phụ thuộc learning rate và cấu trúc bài toán.

Hàm giá trị tối ưu V^*, Q^* dùng chính sách tốt nhất trong lớp đang xét và giả định mô hình. Nếu trạng thái thiếu thông tin, một chính sách tối ưu trên biểu diễn đó có thể kém chính sách dùng bộ nhớ đầy đủ. Từ "tối ưu" luôn cần đi kèm điều kiện: tối ưu đối với trạng thái nào, môi trường nào, đối thủ nào và mục tiêu nào.

3.7 Phương trình Bellman

Với reward kỳ vọng $R(s, a)$ và chuyển trạng thái $P(s' \mid s, a)$, phương trình kỳ vọng Bellman có dạng:

$$V^{\pi}(s) = \sum_{a} \pi(a \mid s) \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^{\pi}(s') \right].$$

Phương trình tối ưu thay trung bình theo chính sách bằng phép cực đại:

$$V^{*}(s) = \max_{a} \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^{*}(s') \right].$$

Đối với Q, ta có:

$$Q^{*}(s,a) = R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) \max_{a'} Q^{*}(s',a').$$

Các biểu thức này làm rõ hai thành phần: lợi ích tức thời và giá trị của trạng thái tiếp theo. Trong một số phần nhập môn của paper, ký hiệu được rút gọn để dễ trình bày. Báo cáo thêm tổng xác suất và điều kiện chính sách để tránh hiểu rằng chỉ cộng mọi giá trị tương lai với trọng số bằng nhau.

Với chân trời hữu hạn, nên viết V_t thay vì một V không phụ thuộc thời gian, và điều kiện biên $V_H = 0$. Khi cùng một quan sát xuất hiện ở đầu và cuối episode, giá trị phần thưởng còn có thể nhận khác nhau. Một bảng Q không chứa thời gian nhưng nhận cả target terminal lẫn nonterminal sẽ trộn các ngữ cảnh này; đây là hạn chế của baseline được giữ lại để phân tích.

3.8 Dynamic programming: biết mô hình để lập kế hoạch

Section IV-A trình bày dynamic programming, gồm đánh giá chính sách, cải thiện chính sách, policy iteration và value iteration. Khi biết mô hình, ta có thể tính kỳ vọng thay vì chờ mẫu thực nghiệm. Policy evaluation giữ nguyên π và giải giá trị; policy improvement chọn hành động tốt hơn theo giá trị hiện có; policy iteration luân phiên hai bước.

Value iteration áp dụng trực tiếp toán tử tối ưu Bellman. Trong MDP hữu hạn có chiết khấu thích hợp, phép lặp có nền tảng hội tụ. Tuy nhiên, khi mô hình lớn hoặc không biết, liệt kê toàn bộ trạng thái và chuyển tiếp có thể không khả thi. Đây là lý do học từ mẫu trở nên cần thiết, chứ không phải vì quy hoạch động vốn kém thông minh hơn RL.

Trong assignment, mô hình chỉ có hai mức nhu cầu, hai hành động và phản ứng rule rõ ràng. Vì thế quy hoạch động hữu hạn chân trời là mốc kiểm tra rất mạnh. Nó giúp phân biệt việc "Q-learning đã học được một chiến lược" với việc "chiến lược đó đạt best response". Một thuật toán học phức tạp mà không so với mốc đơn giản có thể tạo ra kết luận sai về hiệu quả.

Mã DP của báo cáo dùng trạng thái $(d, a^{A}_{t-1}, a^{B}_{t-1})$ và thời điểm t. Với rule opponent, hành động hiện tại $b = d\, a^{A}_{t-1}$. Với mỗi lựa chọn a, thuật toán tính reward kỳ vọng và trung bình giá trị ở hai mức nhu cầu kế tiếp. Việc thực hiện từ $t = H - 1$ về 0 cho ra chính sách phụ thuộc thời gian một cách chính xác trong mô hình.

3.9 Monte Carlo: học từ return hoàn chỉnh

Section IV-B giới thiệu Monte Carlo như cách ước lượng giá trị qua các episode hoàn tất. Với mỗi lần thăm trạng thái, tính return thực tế rồi lấy trung bình. Nếu dữ liệu được lấy đúng từ chính sách mục tiêu và episode phù hợp, trung bình mẫu là cách tự nhiên để ước lượng kỳ vọng. Không cần biết bảng xác suất chuyển trạng thái.

Ưu điểm là không bootstrap từ một giá trị ước lượng khác. Nhược điểm là phải chờ tương lai của episode và return có thể biến động mạnh theo nhiều bước ngẫu nhiên. Nếu một episode dài gồm nhiều thành phần nhiễu, mẫu return ở đầu kỳ có phương sai lớn. First-visit và every-visit phân biệt cách dùng những lần gặp lại trạng thái trong một episode.

Off-policy Monte Carlo cần xử lý khác biệt giữa chính sách thu thập và chính sách đánh giá, thường qua importance sampling với điều kiện hỗ trợ thích hợp. Trọng số có thể rất lớn khi chuỗi dài hoặc chính sách khác nhau mạnh. Do đó việc có log lịch sử không tự động cho phép đánh giá mọi chính sách mới một cách ổn định.

Liên hệ bài lab: mỗi episode có 20 bước nên có thể bổ sung MC để đối chiếu với Q-learning. Tuy vậy, bản thực nghiệm chính không chạy MC; báo cáo không gán cho nó số liệu chưa đo. Đề xuất có thể thực hiện là giữ cùng seed và budget tương tác, thêm MC control với cùng không gian quan sát rồi so mức sai lệch so với oracle.

3.10 Temporal difference và ý nghĩa bootstrapping

Section IV-C trình bày temporal difference, kết hợp việc học từ mẫu với ước lượng giá trị tương lai. TD(0) cho trạng thái có dạng:

$$V(S_t) \leftarrow V(S_t) + \alpha \left[R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \right].$$

Đại lượng trong ngoặc là TD error. Khi target cao hơn dự đoán, giá trị tăng; khi thấp hơn, giá trị giảm. Không cần chờ episode hoàn tất, nhưng target phụ thuộc ước lượng hiện tại nên có thể mang bias. Nhận định "TD phương sai thấp hơn MC" là trực giác thường hữu ích, không phải bất đẳng thức phổ quát cho mọi cấu hình dữ liệu và estimator.

Bootstrapping là dùng một giá trị ước lượng để cập nhật giá trị khác. Trong Q-learning, target có phép max trên trạng thái kế tiếp. Nó truyền thông tin từ tương lai về các hành động trước. Nhưng nếu ước lượng kế tiếp không phù hợp do trạng thái thiếu hoặc môi trường thay đổi, sai lệch cũng có thể truyền ngược.

Ở terminal thật, giá trị tương lai bằng 0. Trong nhiệm vụ continuing bị cắt chỉ để quản lý thời lượng, "hết 20 bước" có thể là truncation và việc bootstrap cần khác terminal. Đề tham chiếu dùng episode hữu hạn với terminal ở bước cuối, nên báo cáo giữ quy ước này; đồng thời chỉ ra cần thêm thời gian vào trạng thái nếu muốn biểu diễn chính xác nhiệm vụ hữu hạn.

3.11 SARSA và Q-learning

SARSA cập nhật theo hành động thực sự được chọn ở trạng thái tiếp theo:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)].$$

Nó gắn với chính sách đang hành động, gồm cả khám phá, nên thường được gọi là on-policy. Q-learning thay $Q(S_{t+1}, A_{t+1})$ bằng $\max_a Q(S_{t+1}, a)$, hướng đến chính sách greedy dù dữ liệu có thể đến từ epsilon-greedy, nên là off-policy trong cách phân loại này.

Sự khác nhau không nói SARSA luôn an toàn còn Q-learning luôn rủi ro. Kết quả phụ thuộc môi trường, reward, biểu diễn và quy trình đánh giá. Ví dụ kinh điển về hành động khám phá nguy hiểm chỉ minh họa một cấu hình cụ thể. Khi thảo luận một ứng dụng kinh doanh, cần đo chi phí khám phá và ràng buộc thực tế.

Ví dụ cập nhật theo đề: $Q = 4$, $\alpha = 0.1$, reward 6, $\gamma = 0.9$, giá trị tốt nhất kế tiếp 5. Target là $6 + 0.9 \times 5 = 10.5$; TD error là 6.5; giá trị mới $4 + 0.1 \times 6.5 = 4.65$. Nếu cùng transition kết thúc thật, target chỉ là 6 và giá trị mới 4.2. Hai ca được kiểm thử riêng trong mã.

3.12 Epsilon-greedy và khám phá

Với hai hành động, epsilon-greedy chọn một hành động ngẫu nhiên trong hai lựa chọn với xác suất ϵ, còn lại chọn hành động tốt nhất theo bảng. Vì nhánh ngẫu nhiên vẫn có thể chọn hành động greedy, xác suất chọn hành động không greedy là $\epsilon/2$ khi chỉ có một greedy action. Nói epsilon bằng đúng tỷ lệ hành động sai sẽ không chính xác.

Lịch tham chiếu giảm theo episode: $\epsilon_e = \max(0.02, 0.2 \times 0.995^e)$ với e bắt đầu từ 0. Giải $0.2 \times 0.995^e \leq 0.02$ cho thấy ngưỡng sàn đạt xấp xỉ episode chỉ số 460. Hơn 4.500 episode còn lại vẫn giữ khám phá nhỏ. Khi đánh giá, báo cáo đặt epsilon bằng 0 và không cập nhật Q.

Khám phá quá ít có thể làm bỏ sót hành động, nhất là trạng thái hiếm. Khám phá quá nhiều gây mất reward trong training và có thể ảnh hưởng hành vi đối thủ. Trong bài bán hàng, một hành động giảm giá hiện tại còn làm rule opponent có khả năng giảm ở bước sau khi nhu cầu cao. Vì vậy chi phí khám phá không chỉ là margin của bước hiện tại.

Báo cáo chạy ba cấu hình khám phá: lịch giảm tham chiếu, epsilon cố định 0.05 và 0.30. Việc giữ cố định learning rate, số episode và các seed giúp so sánh có ý nghĩa. Dù vậy, một chênh lệch sau năm seed chưa đủ chứng minh cơ chế khám phá là nguyên nhân duy nhất; kết quả còn chịu nhiễu Q, aliasing và trộn thời điểm terminal.

3.13 Learning rate và điều kiện hội tụ

Learning rate điều khiển mức thay đổi trước một mẫu mới. Với target cố định y, cập nhật $Q \leftarrow (1 - \alpha)Q + \alpha y$ là một trung bình có trọng số. Alpha lớn phản ứng mạnh, alpha nhỏ tích lũy chậm. Trong thực tế target cũng thay đổi theo Q và dữ liệu nên trực giác trung bình chỉ giải thích một phần hành vi.

Các bảo đảm hội tụ cổ điển cho tabular Q-learning đòi hỏi những điều kiện về môi trường Markov và dừng, độ bao phủ state–action và lịch learning rate thỏa các tổng thích hợp, thường $\sum \alpha_t = \infty$, $\sum \alpha_t^2 < \infty$. Alpha cố định 0.1 và số episode hữu hạn của lab không

đủ để viện dẫn trực tiếp bảo đảm hội tụ về Q^*. [6]

Trong lab, alpha 0.01 có thể cần nhiều mẫu hơn để phản ánh reward; alpha 0.50 có thể nhạy với nhiễu sales và các target terminal thấp. Nhìn một đường reward được làm trơn chưa xác định được Q đã ổn định. Nên kết hợp đánh giá đóng băng, phân phối chính sách qua seed và độ lệch với oracle.

Cũng cần phân biệt ổn định và tối ưu. Một policy luôn giảm giá có thể rất ổn định nhưng lợi nhuận thấp. Một policy tốt có thể cho reward thô dao động lớn vì nhu cầu ngẫu nhiên. Chỉ số thay đổi tham số, tỷ lệ thay đổi hành động và lợi nhuận đánh giá giải đáp những câu hỏi khác nhau.

3.14 Actor–critic và policy gradient

Section IV-D giới thiệu actor–critic, nơi actor biểu diễn chính sách còn critic ước lượng chất lượng để hướng dẫn actor. Một dạng mục tiêu là $J(\theta) = \mathbb{E}_{\pi_\theta}[G_0]$. Gradient có thể được ước lượng bằng log xác suất hành động nhân một tín hiệu lợi thế. Critic giúp giảm phương sai hoặc cung cấp tín hiệu thường xuyên hơn.

Actor–critic hữu ích khi hành động liên tục hoặc không gian quá lớn cho bảng Q. Tuy nhiên, nó thêm mô hình, learning rate và quan hệ phụ thuộc giữa hai quá trình học. Critic sai có thể hướng actor sai; actor thay đổi làm dữ liệu critic học thay đổi. Đây không phải phép thay thế mặc nhiên tốt hơn cho bài có tám giá trị Q.

Một số câu trong phần thảo luận của paper nói về hội tụ ở mức tổng quát. Báo cáo đọc thận trọng: policy gradient không được bảo đảm tìm global optimum cho mọi bài toán mạng nơ-ron và mọi ước lượng gradient; model-based cũng không được bảo đảm đúng mô hình khi lớp mô hình sai. Cần đặt các phát biểu về hội tụ trong giả định toán học cụ thể.

Liên hệ cải tiến: nếu mở rộng hành động thành mức chiết khấu liên tục và thêm tồn kho, có thể cân nhắc actor–critic hoặc phương pháp điều khiển tối ưu. Trước đó cần một môi trường được hiệu chỉnh và baseline đủ mạnh. Dùng mạng sâu sớm trong môi trường toy làm tăng khó khăn giải thích mà chưa có lý do về biểu diễn.

3.15 Bayesian methods và bất định

Section IV-E đề cập phương pháp Bayesian, dùng phân phối niềm tin để biểu diễn điều chưa biết. Thay vì giữ một con số reward trung bình duy nhất, có thể giữ posterior cho tham số xác suất chuyển đổi rồi chọn hành động theo bất định. Thompson sampling là một ví dụ về lấy mẫu giả thuyết để cân bằng khám phá và khai thác trong các bài phù hợp.

Cần tách **aleatoric uncertainty** do tính ngẫu nhiên của môi trường và **epistemic uncertainty** do thiếu hiểu biết. Dù biết xác suất mua chính xác, sales binomial vẫn ngẫu nhiên. Thu thập thêm dữ liệu giúp giảm bất định tham số nhưng không xóa tính ngẫu nhiên của từng lượt mua. Nhầm hai loại dễ dẫn đến yêu cầu một policy phải thắng trong mọi episode.

Trong bài bán hàng, nếu mô hình conversion chưa biết, có thể dùng posterior cho từng ngữ cảnh và hành động. Nhưng đối thủ phản ứng qua lịch sử khiến mô hình bandit độc lập

có thể thiếu động lực dài hạn. Một cải tiến cần xác định liệu đang học reward tức thời hay toàn bộ quá trình quyết định có trạng thái.

3.16 Deep Q-Network và các thành phần ổn định

Section IV-F trình bày DQN như cách dùng mạng nơ-ron xấp xỉ Q cho không gian trạng thái lớn. Hai thành phần nổi bật là experience replay và target network. Replay tái sử dụng transition và giảm tương quan theo thời gian trong minibatch; target network làm mục tiêu thay đổi chậm hơn so với việc dùng cùng một mạng cập nhật ngay lập tức.

Một target điển hình có dạng $y = r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a')$ cho transition không terminal, rồi tối ưu bình phương sai số giữa y và $Q_\theta(s, a)$. DQN vẫn chịu vấn đề xấp xỉ hàm, tối ưu không lồi và dữ liệu phụ thuộc chính sách. Thêm replay không tự khắc phục non-stationarity khi nhiều agent cùng học.

Với tám giá trị Q trong lab, mạng sâu sẽ có nhiều tham số hơn đại lượng cần học và tốn thêm tuning. Bảng Q là lựa chọn hợp lý để người học nhìn rõ từng giá trị, từng trạng thái và từng lỗi. Bài học từ paper là chọn công cụ theo kích thước và đặc tính bài toán, không chọn thuật toán mới chỉ vì có vẻ mạnh.

3.17 So sánh các họ phương pháp

Phương pháp	Cần mô hình chuyển tiếp?	Target chính	Điểm mạnh	Hạn chế chính
Dynamic programming	Có	Kỳ vọng Bellman	Mốc chính xác khi mô hình nhỏ, đúng	Chi phí không gian trạng thái, sai mô hình
Monte Carlo	Không	Return sau episode	Không bootstrap, diễn giải rõ	Chờ cuối kỳ, phương sai return
TD/SARSA	Không	Reward và giá trị hành động kế tiếp	Cập nhật online, theo policy hành động	Bootstrap và phụ thuộc biểu diễn
Q-learning	Không	Reward và max Q kế tiếp	Tabular nhỏ dễ triển khai	Không bảo đảm ở mọi môi trường, nhạy trạng thái thiếu
Actor–critic	Không nhất thiết	Gradient policy và critic	Phù hợp policy tham số, hành động lớn	Nhiều nguồn bất ổn và tham số
Bayesian	Tùy thiết kế	Posterior và quyết định dưới bất định	Biểu diễn điều chưa biết	Chi phí suy luận, giả định prior/model
DQN	Không cần mô hình tường minh	Bellman target từ mạng mục tiêu	Xấp xỉ Q trên đầu vào lớn	Data hungry, khó giải thích, cần ổn định

Các trục phân loại không hoàn toàn loại trừ nhau. Bayesian có thể dùng cho model-based hoặc một thành phần reward; actor–critic có thể dùng mô hình hoặc không; deep RL có thể value-based hoặc policy-based. Do vậy không nên vẽ một cây phân loại duy nhất rồi coi mỗi thuật toán chỉ được đặt vào đúng một lá.

3.18 Ứng dụng trong paper: đọc bằng cấu trúc bài toán

3.18.1 Healthcare

Paper xem xét lựa chọn điều trị tuần tự, cá nhân hóa và một số quy trình hỗ trợ. Điểm phù hợp với RL là quyết định hiện tại ảnh hưởng sức khỏe và lựa chọn ở tương lai. Nhưng việc quan sát dữ liệu điều trị lịch sử không tương đương có quyền khám phá trực tiếp trên bệnh nhân. Nhiễu quan sát, confounding và thiếu độ bao phủ hành động khiến đánh giá chính sách khó.

Báo cáo chỉ dùng miền healthcare ở **phân loại yêu cầu hành chính**, không tạo chính sách điều trị, chẩn đoán hay khuyến nghị thuốc. Liên hệ này giúp nhận ra khác biệt mức rủi ro giữa một ví dụ nhập môn và ứng dụng clinical mà paper khảo sát. Các chi tiết y khoa thực hành nằm ngoài phạm vi bộ bài.

3.18.2 Robotics và điều khiển tự động

Robot có chuỗi quan sát–hành động liên tục và hậu quả động học, nên RL là khung tự nhiên. Phần thưởng có thể gắn với hoàn thành nhiệm vụ, tiêu hao năng lượng hoặc tránh va chạm. Khó khăn là tương tác thật tốn kém và hành động sai có thể gây hỏng vật lý. Mô phỏng giúp thu thập dữ liệu nhưng có khoảng cách sim-to-real.

Một thí nghiệm robot tốt không chỉ báo reward trung bình; cần tỷ lệ thành công, vi phạm giới hạn, độ bền qua điều kiện tải và khả năng khôi phục. Các chỉ số này có thể được chuyển thành tư duy cho tutorial: một hệ trả lời nhanh nhưng không giữ bất biến ngân sách vẫn thất bại nhiệm vụ.

3.18.3 Communication và networking

Ứng dụng gồm điều khiển tài nguyên, lập lịch, đường truyền hoặc cấu hình. Trạng thái mạng biến động và quyết định cục bộ có thể ảnh hưởng người dùng khác. RL phù hợp khi có tương tác lặp và feedback, nhưng không thể bỏ qua độ trễ quyết định và sự thay đổi phân phối lưu lượng.

Nếu nhiều node học độc lập, throughput của một node phụ thuộc chính sách của các node còn lại. Khi mọi node cập nhật, mỗi node quan sát một môi trường hiệu dụng đang đổi. Đây là cầu nối trực tiếp đến vấn đề non-stationarity trong câu 8 của assignment.

3.18.4 NLP

Paper thảo luận ứng dụng RL trong các tác vụ ngôn ngữ, đối thoại và tạo chuỗi, nơi đầu ra dài và tiêu chí cuối khó chia thành nhãn cho từng bước. Phần thưởng có thể là độ hoàn thành mục tiêu hoặc đánh giá chất lượng. Tuy nhiên, một reward dễ tối ưu có thể khác mục tiêu con người thực sự muốn.

Cần giữ bối cảnh thời gian: bài công bố năm 2020 và không phải survey về toàn bộ các kỹ thuật căn chỉnh LLM xuất hiện sau đó. Báo cáo dùng nó cho nền tảng RL và không gán các mô hình mới vào danh sách đóng góp nguyên bản. So sánh với paper MAS năm 2025 cần nêu khoảng cách này.

3.18.5 Autonomous IoT và computer vision

Trong IoT, bài toán thường liên quan năng lượng, kết nối, thu thập dữ liệu và phân bổ tài nguyên. Hành động có thể làm thay đổi thời lượng pin hoặc mức thông tin nhận được. Với computer vision, RL có thể tham gia lựa chọn vùng quan sát hoặc quyết định tuần tự, trong khi nhiều bài nhận dạng thuần túy vẫn thích hợp hơn với supervised learning.

Bài học lựa chọn phương pháp: có ảnh hoặc có sensor chưa đủ để kết luận cần RL. Nếu nhiệm vụ chỉ dự đoán nhãn từ một ảnh và hành động không ảnh hưởng dữ liệu tương lai, classifier có thể phù hợp hơn. RL được biện minh bởi cấu trúc quyết định và phản hồi, không phải loại dữ liệu đầu vào.

3.18.6 Games, self-organized systems và scheduling

Trò chơi cung cấp môi trường có luật rõ, vòng tương tác lặp và công cụ đo reward, vì vậy thuận tiện cho nghiên cứu. Tuy nhiên, thành công trong trò chơi có thông tin sạch không đồng nghĩa thành công trong hệ kinh doanh có nhu cầu thay đổi và hạn chế can thiệp. Cần kiểm tra cơ chế chuyển từ benchmark sang miền ứng dụng.

Trong scheduling, quyết định hiện tại có thể chiếm tài nguyên và ảnh hưởng các công việc về sau. Ràng buộc cứng như deadline hoặc năng lực máy cần được thể hiện trực tiếp. Nếu chỉ thêm một penalty nhỏ vào reward, agent có thể chọn vi phạm để đổi lợi ích khác. Điều này tương tự việc phải lọc sản phẩm vượt ngân sách trước khi xếp hạng trong tutorial.

3.19 Các thách thức lớn và đọc phản biện

3.19.1 Sample efficiency và độ bao phủ

Paper nhấn mạnh chi phí dữ liệu và không gian trạng thái lớn. Đối với lab, 5.000 episode tương đương 100.000 bước mỗi lượt training, nhưng không có nghĩa mọi state–action được quan sát đủ cân bằng. Một trạng thái chứa đối thủ giảm giá có thể hiếm khi policy tiến về không giảm giá. Chỉ báo tổng số bước che khuất sự thiếu bao phủ cục bộ.

Đánh giá sample efficiency nên xem hiệu năng theo số tương tác và dùng cùng ngân sách giữa thuật toán. So sánh một agent được train gấp mười lần với baseline ít dữ liệu sẽ

không tách được chất lượng phương pháp. Một đề xuất mở rộng là lưu visit count cho từng state–action và phân tích sai lệch Q ở các ô hiếm.

3.19.2 Reward design và reward hacking

Mục tiêu được số hóa có thể bỏ sót điều người dùng quan tâm. Nếu reward là số đơn, agent dễ ưu tiên giảm giá mạnh dù lợi nhuận thấp. Nếu reward là doanh thu, nó có thể bán hàng lỗ. Nếu reward chỉ là lợi nhuận ngắn hạn, nó có thể không tính trải nghiệm khách hàng hoặc biến động giá. Việc định nghĩa reward là một phần nội dung khoa học, không chỉ tham số code.

Cải tiến phù hợp là tách objective và constraints: tối đa lợi nhuận trong miền hành động tuân thủ các điều kiện về margin và ngân sách. Có thể đưa một số ưu tiên mềm vào reward, nhưng phải công bố trọng số và phân tích độ nhạy. Một con số tổng hợp cao không chứng minh mọi mặt đều được cải thiện.

3.19.3 Partial observability và non-stationarity

Khi chỉ quan sát một phần, agent có thể cần lịch sử hoặc belief state. Trong lab, rule opponent dùng hành động trước của agent A, nhưng bảng Q của A chỉ nhận nhu cầu và hành động trước của B. Hai lịch sử khác nhau có cùng quan sát A nhưng dẫn đến hành động hiện tại khác nhau của B. Đây là ví dụ cụ thể về state aliasing, ngay cả khi đối thủ không học.

Khi cả hai agent cùng học, chính sách đối thủ thay đổi qua episode nên reward và chuyển tiếp hiệu dụng đối với một learner thay đổi. Frozen evaluation sau training giúp đo chất lượng cặp policy tại một thời điểm, nhưng không chứng minh quá trình học đã tiến đến cân bằng. Cần thêm cross-play, nhiều seed và phân tích dao động hoặc best response.

3.19.4 Bias–variance và những phát biểu cần điều kiện

Phần Challenges của paper so sánh MC, TD và biến động ước lượng. Báo cáo giữ trực giác rằng target dài có thể nhiều nhiễu hơn, còn bootstrap có thể mang sai lệch, nhưng không biến trực giác đó thành kết luận tuyệt đối. Một estimator cụ thể có thể có hành vi khác tùy dữ liệu và cấu trúc môi trường.

Paper cũng có những lời khẳng định rộng về dữ liệu tổng hợp và giảm bias. Dữ liệu tổng hợp không bảo đảm loại thiên lệch; nó có thể tái tạo hoặc làm nặng thiên lệch mô hình sinh. Trong tutorial, classifier đạt 100% trên test tổng hợp nhưng vẫn nhận sai hai câu ngoài miền. Đây là minh chứng thực tế vì sao cần tách chất lượng trên phân phối tạo dữ liệu khỏi độ bền ngoài phân phối.

3.20 Kiểm tra giả định trên chính môi trường assignment

Gọi trạng thái đầy đủ $x_t = (d_t, u_t, v_t)$ với $u_t = a^A_{t-1}$ và $v_t = a^B_{t-1}$. Với rule opponent, $b_t = d_t u_t$. Quan sát tham chiếu của A là $o_t = (d_t, v_t)$, không có u_t. Xét $d_t = 1, v_t = 0$: nếu $u_t = 0$ thì B

chọn 0; nếu $u_t = 1$ thì B chọn 1. Cùng một quan sát và cùng hành động A dẫn đến reward kỳ vọng khác nhau.

Ví dụ A không giảm: ở nhu cầu cao, B không giảm cho conversion 0.55 và lợi nhuận kỳ vọng $20 \times 0.55 \times 5 = 55$; B giảm cho conversion 0.45 và kỳ vọng 45. Chênh 10 không phải nhiễu ngẫu nhiên, mà do thông tin thiếu. Vì A biết hành động mình vừa làm, việc bổ sung bit này là khả thi và chi phí nhỏ.

Một yếu tố khác là thời điểm. Ở bước cuối, Q chỉ gồm reward ngay; ở bước đầu còn nhiều reward tương lai. Nếu dùng một bảng cho cả hai mà không có thời gian, cùng state–action nhận target theo những chân trời khác nhau. Báo cáo giữ baseline tám giá trị để đúng tinh thần đề, rồi dùng DP với đầy đủ trạng thái và thời gian để kiểm tra.

3.21 Phân tích lợi nhuận kỳ vọng và chính sách mốc

Với $a, b \in \{0, 1\}$, conversion ở các tham số mặc định luôn nằm giữa 0.20 và 0.70 nên clip không hoạt động. Reward kỳ vọng là:

$$\bar{r}(d, a, b) = (10 + 10d)(0.30 + 0.25d + 0.15a - 0.10b)(5 - 2a).$$

Nếu $b = 0$, nhu cầu thấp: không giảm thu 15, giảm thu 13.5; nhu cầu cao: không giảm 55, giảm 42. Nếu $b = 1$, các giá trị lần lượt là 10 và 10.5 ở thấp, 45 và 36 ở cao. Vì rule opponent chỉ giảm khi nhu cầu cao và A vừa giảm ở bước trước, trường hợp thấp mà B giảm không xuất hiện trong baseline.

Không giảm là tốt hơn tức thời ở các trạng thái mà rule opponent tạo ra. Nó còn tránh kích hoạt phản ứng giảm giá của B ở bước kế tiếp khi nhu cầu cao. Quy hoạch động xác nhận hành động 0 ở mọi thời điểm và trạng thái đầy đủ trong mô hình tham chiếu. Đây là best response với đối thủ đã định nghĩa, không phải chứng minh luôn không giảm là tối ưu cho mọi thị trường.

Khi cả hai khởi đầu với lịch sử 0 và dùng rule, không bên nào tạo ra tín hiệu giảm giá nên cả hai luôn chọn 0. Nhu cầu thấp/cao có xác suất 1/2, vì vậy lợi nhuận kỳ vọng mỗi bước là $(15 + 55)/2 = 35$. Kết quả đánh giá thực nghiệm dao động quanh 35 do nhu cầu và sales; kỳ vọng 35 không được chép thành mọi số đo của bảng kết quả.

3.22 Cầu nối với MAS và phương pháp nghiên cứu chung

Paper MAS nhấn mạnh kênh phối hợp, còn paper RL nhấn mạnh điều chỉnh chính sách bằng feedback. Hai quan điểm bổ sung: một hệ có thể thiết kế cơ chế giao tiếp rất rõ nhưng policy quyết định kém; một learner có reward tốt trong mô phỏng vẫn có thể bị tích hợp sai vào một workflow. Cần kiểm tra cả vòng học và hợp đồng vận hành.

Ví dụ nếu chuyển learner bán hàng thành một thành phần của hệ thương mại, nó chỉ nên đề xuất mức giảm trong tập được phép. Agent chính sách kiểm tra margin, inventory xác nhận khả năng giao hàng và hệ giao dịch thực hiện với xác nhận tương ứng. Reward training không tự cấp quyền thay đổi giá hoặc bỏ qua dữ liệu kho.

Một khuôn nghiên cứu chung là: mô hình hóa giả định; xây baseline đơn giản; lưu dữ

liệu thô; đánh giá đóng băng; phân tích theo loại trạng thái; thử thay đổi môi trường; và nêu giới hạn suy rộng. Cách làm này có giá trị hơn kết luận "AI học được nên thông minh hơn luật" từ một lượt chạy.

3.23 Kết luận riêng cho Paper 2

Paper cung cấp đường đi từ MDP đến các họ thuật toán và ứng dụng. Phần có giá trị cho bài học là những câu hỏi nền tảng: trạng thái có đủ không, reward đo gì, khám phá tốn gì, target được tính thế nào và kết quả có được đánh giá độc lập không? Khi áp dụng vào assignment, các câu hỏi này phát hiện những vấn đề cụ thể mà việc chỉ chạy chương trình tham chiếu sẽ bỏ qua.

Báo cáo bổ sung ký hiệu kỳ vọng rõ, phân biệt Markov với tính dừng, hữu hạn chân trời với continuing, hiệu năng training với evaluation và tối ưu với hội tụ. Các thực nghiệm đi kèm không nhằm thay thế survey, mà biến kiến thức tổng quan thành một trường hợp có thể tính tay, chạy lại và phản biện bằng dữ liệu.

4 Chuyên đề toán học và kỹ thuật bổ sung

Phần này phát triển các phép tính và thiết kế cụ thể để làm rõ hai paper. Đây là phân tích của báo cáo trên mô hình được cung cấp, không phải các định lý hoặc kết quả thực nghiệm mới được gán cho tác giả hai survey. Các suy luận đi từ giả định đến hệ quả, giúp kiểm tra tính nhất quán giữa mô hình, code và kết luận.

4.1 1. Từ collaboration channel đến hợp đồng có thể kiểm

Giả sử một ranker trả ba sản phẩm, mỗi sản phẩm có ID, score và lời giải thích. Coordinator muốn trả kết quả cho người dùng cần kiểm ít nhất bốn thuộc tính: ID có trong catalog; ID đến từ tập candidate của truy vấn hiện tại; giá nằm trong ngân sách; kho đã được đọc với tuổi dữ liệu hợp lệ. Score cao không thay thế bất kỳ điều kiện nào trong bốn điều kiện đó.

Có thể biểu diễn một proposal z bằng predicate:

$$\text{Valid}(z) = \text{KnownID}(z) \wedge \text{Retrieved}(z, q) \wedge \text{Affordable}(z, B) \wedge \text{FreshStock}(z, \tau).$$

Các predicate dùng dữ liệu từ những nguồn có thẩm quyền khác nhau. Catalog xác nhận ID và price; retrieval trace xác nhận candidate provenance; inventory xác nhận stock và thời điểm. Coordinator không cần làm lại mọi công việc chuyên môn, nhưng phải biết nguồn nào quyết định thuộc tính nào. Nếu output một agent nói "đã kiểm tra tất cả" mà không có dữ liệu xác nhận, chuỗi kiểm chứng bị đứt.

Ví dụ một message giả trả p99 với giá 0 có thể vượt qua kiểm budget nhưng thất bại KnownID. Một message trả p1 đúng catalog nhưng không thuộc truy vấn hoặc vượt budget lại thất bại điều kiện khác. Việc dùng nhiều predicate ngăn một dấu hiệu bề ngoài hợp lệ che mất sai sót ở thuộc tính khác. Đây là cách cụ thể hóa ý tưởng kênh cộng tác trong paper MAS.

4.2 2. State machine cho workflow thay vì vòng chat không giới hạn

Một quy trình mua hàng có thể có các trạng thái RECEIVED, SEARCHED, RANKED, VALIDATED, STOCK_VERIFIED và COMPLETED. Nhánh lỗi gồm INVALID_INPUT, NO_RESULTS, TIMEOUT, CONTRACT_ERROR và AVAILABILITY_UNVERIFIED. Điều kiện chuyển trạng thái phải được định nghĩa trước: chỉ response đúng envelope mới cho phép sang bước sau; chỉ dữ liệu kho mới và đủ ID mới cho phép COMPLETED có claim verified.

Bất biến ở cấp state machine có thể phát biểu: mọi item trong COMPLETED đã đi qua VALIDATED và STOCK_VERIFIED trong cùng task ID. Một response cũ đến sau timeout không được hồi sinh yêu cầu đã kết thúc thành thành công, trừ khi thiết kế explicitly cho phép một phiên mới. Tutorial không có queue response muộn vì lời gọi là coroutine cục bộ, nhưng task ID đã tạo ranh giới phù hợp để mở rộng.

Lợi ích của state machine là phân biệt lỗi chuyên môn và lỗi giao tiếp. Empty search là một kết quả hợp lệ của CatalogAgent; timeout là không có kết quả đúng hạn; malformed là

vi phạm hợp đồng. Nếu tất cả đều chuyển thành danh sách trống không có status, người sử dụng và nhà phát triển không thể biết nên sửa truy vấn, thử lại hay kiểm tra dữ liệu.

4.3 3. Complexity của giao tiếp và đường găng

Với n tác tử và r vòng trao đổi, một topology đầy đủ có hướng có thể tạo $r\,n(n-1)$ message. Cấu trúc tập trung gọi mỗi worker một lần theo request–response có khoảng $2n$ message. Nhưng số message chỉ là một phần chi phí. Nếu mỗi message lặp một context dài L token, tổng token có thể tăng theo L nhân số lượt, cộng với output và phần tính toán của mô hình.

Một mô hình chi phí do báo cáo đề xuất là:

$$C_{total} = \sum_{v \in V} C_{compute}(v) + \sum_{e \in E} C_{message}(e) + C_{retry} + C_{verification}.$$

Thời gian hoàn tất có thể gần độ dài đường găng trong DAG cộng scheduling overhead. Nếu catalog và dữ liệu chính sách độc lập, hai thao tác có thể chạy cùng lúc. Nếu inventory chỉ nhận danh sách sau xếp hạng, nó nằm sau ranker. Việc chạy mọi agent song song mà chưa có input hợp lệ không phải tối ưu hợp lý.

Một baseline công bằng nên được cấp cùng dữ liệu và quyền công cụ. Nếu MAS chạy song song trên bốn CPU còn baseline chỉ một CPU, chênh lệch latency trộn kiến trúc với tài nguyên. Nếu cả hai đều dùng dịch vụ ngoài, cần cùng timeout, pool và cache. Bảng cục bộ của tutorial ghi rõ giới hạn này, đặc biệt baseline timeout được tiêm đồng bộ.

4.4 4. Mô hình lỗi dây chuyền và giá trị kiểm chứng

Giả sử một pipeline có k bước bắt buộc và mỗi bước thành công độc lập với xác suất p_i. Xác suất mọi bước thành công là $\prod_i p_i$. Nếu bốn bước đều đạt 0.95 thì tích là khoảng 0.8145, không còn 0.95. Đây chỉ là ví dụ giả định độc lập để giải thích vì sao nhiều thành phần tạo thêm rủi ro; hệ thật có thể có tương quan lỗi và fallback.

Một validator có thể phát hiện tỷ lệ d của lỗi nhưng cũng có false rejection. Giá trị của nó phụ thuộc chi phí sai sót và chi phí từ chối. Nếu lỗi tạo claim tồn kho sai có hậu quả đáng kể, chặn một kết quả chưa xác minh có thể là lựa chọn đúng dù làm giảm số danh sách được trả. Vì vậy success không nên định nghĩa chỉ bằng tỷ lệ có output không rỗng.

Điểm kiểm chứng tốt nhất nằm ở nơi có thông tin độc lập. Một LLM reviewer đọc cùng output không có catalog khó phát hiện p99 là bịa. Một phép kiểm membership trong tập ID thực hiện điều đó xác định. Khi nhiệm vụ có công cụ kiểm chứng chính xác, cần tận dụng thay vì chỉ thêm một vai trò hội thoại.

4.5 5. Đa số và tương quan sai số

Với ba bộ trả lời độc lập có cùng xác suất đúng p, đa số đúng khi đúng cả ba hoặc đúng đúng hai. Kết quả $p^3 + 3p^2(1-p)$ tăng so với p nếu $p > 0.5$ trong mô hình nhị phân này. Nếu $p < 0.5$, đa số có thể làm kết quả tệ hơn. Vì vậy ngay mô hình độc lập đơn giản cũng đòi hỏi chất lượng thành viên đủ tốt.

Trong hệ LLM, lỗi không độc lập: các thành viên có thể chung backbone, dữ liệu hoặc prompt; sau vòng đầu, chúng đọc câu trả lời của nhau và càng tương quan. Thêm vòng tranh luận có thể cung cấp phản biện, nhưng cũng có thể gây anchoring. Một thí nghiệm tốt cần đo cấu hình độc lập trước trao đổi và sau trao đổi, không chỉ báo cuối cùng có đa số.

Đa dạng hữu ích có thể đến từ công cụ kiểm khác nhau hoặc nguồn dữ liệu độc lập. Người tính bằng Python và người kiểm đơn vị vật lý có khả năng phát hiện những nhóm lỗi khác nhau. Đa dạng persona không đủ để suy ra đa dạng năng lực. Đây là điểm quan trọng khi chuyển taxonomy role-based thành thiết kế nghiên cứu thực tế.

4.6 6. Confusion matrix, macro-F1 và selective prediction

Confusion matrix M_{ij} đếm số ví dụ có nhãn thật i nhưng dự đoán j. Precision lớp i là $M_{ii}/\sum_j M_{ji}$; recall là $M_{ii}/\sum_j M_{ij}$; F1 là trung bình điều hòa của hai chỉ số. Macro-F1 lấy trung bình đều các lớp, hữu ích khi không muốn lớp đông áp đảo chỉ số tổng.

Trong dataset tutorial, ba lớp có cùng 36 câu test và toàn bộ đường chéo là 36, nên accuracy và macro-F1 đều bằng 1. Kết quả này được tính từ predictions CSV, không suy ra từ training loss. Một mô hình có loss nhỏ không tự động được phép gọi là đạt 100% nếu chưa so dự đoán với nhãn held-out.

Với ngưỡng τ, coverage là tỷ lệ ví dụ được chấp nhận và selective risk là tỷ lệ sai trong tập chấp nhận. Một hệ có thể tăng độ đúng bằng cách từ chối nhiều hơn; cần báo đồng thời coverage. Tuy nhiên, các đường coverage–risk trên in-domain validation không mô tả đầy đủ false acceptance OOD. Sáu probe ngoài miền của tutorial cho thấy một câu xa nhiệm vụ vẫn có thể có confidence trên 0.9.

4.7 7. Vì sao chia theo nhóm dữ liệu quan trọng

Nếu sinh nhiều câu bằng một template rồi chia ngẫu nhiên từng câu, train và test có thể chỉ khác một hậu tố nhỏ. Mô hình dễ khai thác mẫu bề mặt và test trở nên lạc quan. Chia theo họ động từ trong tutorial làm các pattern mở đầu test không có nguyên dạng trong train, giảm một loại rò rỉ.

Dù vậy, các đối tượng như laptop, bảo tàng hoặc giấy tờ bảo hiểm vẫn lặp giữa split; vocabulary chỉ xây trên train nhưng các token nổi bật của từng class vẫn xuất hiện. Đây là lý do kết quả không được gọi là tổng quát hóa ngôn ngữ tự nhiên rộng. Một split khó hơn có thể giữ riêng cả nhóm đối tượng hoặc dùng câu người dùng viết độc lập.

Tập validation được dùng chọn epoch và threshold; test dùng đo cuối. Nếu sau khi đọc các lỗi OOD ta sửa lexicon, các câu đã xem trở thành dữ liệu development. Muốn báo hiệu năng mới đáng tin, phải tạo test khác. Tính độc lập là thuộc tính của quy trình nghiên cứu, không chỉ của tên tệp train/test.

4.8 8. Chứng minh observation baseline chưa đủ Markov

Xét hai lịch sử có cùng $d_t = 1$ và hành động trước của B bằng 0. Lịch sử thứ nhất có $a^A_{t-1} = 0$, thứ hai có $a^A_{t-1} = 1$. Theo rule B, hành động hiện tại lần lượt là 0 và 1. A không

giảm trong cả hai trường hợp. Khi đó reward kỳ vọng A là 55 hoặc 45. Cùng observation và action nhưng khác luật reward có điều kiện theo phần lịch sử bị bỏ.

Kết luận không phải "mọi Q-learning ở đây vô nghĩa". Bảng Q vẫn có thể học một chính sách tốt trong lớp phản ứng theo observation. Nhưng không thể áp nguyên các giả định tabular MDP đã đủ trạng thái. Việc thêm own previous action là một cách sửa có lý do rõ, vì A vốn có thể ghi nhớ hành động của chính mình mà không cần cảm biến mới.

Nếu opponent thay luật dựa trên cả mười bước quá khứ, một bit lịch sử có thể vẫn chưa đủ. Khi đó cần state augmentation lớn hơn, belief hoặc mô hình đối thủ. Chọn state là quá trình cân bằng đủ thông tin và chi phí biểu diễn, không phải chỉ chọn vector ngắn để code thuận tiện.

4.9 9. Hữu hạn chân trời và terminal aliasing

Trong nhiệm vụ 20 bước, cùng một trạng thái kinh tế ở bước 1 và bước 20 có số bước tương lai khác nhau. Một hàm giá trị chính xác nên có chỉ số t. Ở bước 20, return chỉ gồm reward hiện tại; ở bước 1 còn 19 reward sau đó. Nếu bảng Q không chứa thời gian nhưng update cuối dùng target không bootstrap, nó nhận các target không đồng nhất cho cùng ô.

Có hai cách mô hình hóa hợp lệ tùy ý nghĩa bài toán. Nếu 20 bước là horizon thật, thêm thời gian và terminal ở cuối. Nếu thị trường continuing còn 20 chỉ là cách cắt log, có thể bootstrap qua điểm cắt thích hợp thay vì xem đó là terminal kinh tế. Đề tham chiếu dùng cách đầu nhưng biểu diễn không có thời gian; báo cáo giữ để đối chiếu rồi bổ sung oracle đầy đủ.

Sửa terminal mà không hiểu mục tiêu có thể gây lỗi khác. Một số bài thật có thanh lý tồn kho cuối kỳ, khi đó reward ở terminal còn bao gồm giá trị tồn và nghĩa vụ chưa hoàn tất. Chỉ gán future value 0 mà bỏ các phần đó cũng là mô hình hóa sai. Vì vậy thảo luận terminal cần bắt đầu từ bài toán, không từ công thức code.

4.10 10. Quy hoạch động lùi: một ví dụ hai bước

Dùng horizon hai bước với rule opponent và gamma 0.9. Ở bước cuối, khi nhu cầu thấp thì B không giảm; action 0 có reward kỳ vọng 15 so với 13.5. Khi nhu cầu cao, B có thể là 0 hoặc 1; action 0 lần lượt đạt 55 hoặc 45, đều lớn hơn 42 hoặc 36 của action 1. Do đó hành động cuối tối ưu là 0 ở mọi state.

Ở bước đầu, chọn action 0 làm own previous của bước sau bằng 0, nên rule B không giảm bước sau. Giá trị tối ưu kỳ vọng bước sau là 35. Chọn action 1 làm B giảm nếu nhu cầu sau cao; giá trị khi A chọn 0 ở bước sau là $(15 + 45)/2 = 30$. Như vậy action 1 còn mất thêm $0.9(35 - 30) = 4.5$ giá trị tương lai so với action 0, ngoài chênh reward tức thời.

Lập luận này cho thấy cơ chế dài hạn trong mô hình. Nhu cầu độc lập không làm bài toán thành các lựa chọn tức thời độc lập vì opponent phản ứng theo lịch sử action. Với horizon 20, DP lùi kiểm toàn bộ state/time và cho cùng kết luận hành động 0. Đây là ví dụ liên hệ trực tiếp Bellman với tương tác đa tác tử.

4.11 11. Giá trị kỳ vọng của policy bằng forward occupancy

Để kiểm tra số đo mà không cần thêm random seeds, báo cáo triển khai một phép tính chính xác cho policy cố định. Gọi $\mu_t(x)$ là xác suất full state $x = (d, u, v)$ ở bước t. Khởi đầu $\mu_0(0,0,0) = \mu_0(1,0,0) = 1/2$. Với action $a = \pi(d, v)$ và $b = d\,u$, trạng thái sau là (d', a, b), trong đó mỗi d' có xác suất $1/2$.

Cập nhật occupancy:

$$\mu_{t+1}(d', a, b) \mathrel{+}= \frac{1}{2}\mu_t(d, u, v).$$

Reward kỳ vọng mỗi bước là $\sum_x \mu_t(x)\bar{r}(x, \pi(x), b(x))$; trung bình qua 20 bước cho exact policy mean không chiết khấu. Đây là đánh giá một policy bằng forward probability, khác với DP lùi tìm policy tốt nhất. Cả hai dùng cùng full state để giữ đúng phản ứng của đối thủ.

Tệp `assignment/analyze_exact.py` áp dụng phép tính cho mọi Q-table đã lưu. Policy bits theo thứ tự observation (0,0), (0,1), (1,0), (1,1) giúp đọc hành vi. Một policy có bit 1 ở state không thể tới có thể vẫn đạt cùng mean 35 như policy 0000. Bởi vậy so bảng policy từng ô không thay thế phân tích trạng thái thực sự được thăm.

4.12 12. Tách lỗi học khỏi nhiễu evaluation bằng exact values

Baseline seed 11 và 55 có policy bits 0000, exact mean 35. Seed 44 có 0001 nhưng state gây discount không xuất hiện từ điều kiện khởi đầu khi policy không tạo discount trước, nên exact mean cũng 35. Seed 22 và 33 có 1000, exact mean xấp xỉ 32.448. Lợi nhuận thấp hơn ở hai seed sau là khác biệt policy thực sự, không chỉ nhiễu test.

Lượt minh họa seed 42 có exact mean xấp xỉ 25.698, trong khi evaluation sample mean là 25.5591. Điều này xác nhận policy snapshot thật sự kém trong mô hình chứ không đơn thuần gặp một tape rất xấu. Sự khác biệt nhỏ giữa exact và sampled mean là nhiễu hữu hạn; khoảng cách lớn với 35 chủ yếu nằm ở policy đã học.

Bảng exact cho toàn bộ 25 cấu hình/seed được xuất trong `exact_policy_values.csv`. Phép tính này chỉ khả thi vì môi trường nhỏ và biết đầy đủ reward/transition. Trong ứng dụng thật, không thể gọi một mô hình chưa hiệu chỉnh là exact oracle; độ đúng khi ấy bị giới hạn bởi sai số mô hình.

4.13 13. Kỳ vọng và phương sai profit ở một bước

Nếu action và demand cố định, $S \sim \text{Binomial}(N, p)$ và margin là m, thì $R = mS$. Vì vậy $\mathbb{E}[R] = mNp$ và $\text{Var}(R) = m^2Np(1-p)$. Với không giảm và demand thấp, $N = 10, p = 0.3, m = 5$, ta có mean 15 và variance 52.5. Demand cao với $p = 0.55$ cho mean 55 và variance 123.75.

Nếu demand thấp/cao đều xác suất 1/2, dùng luật phương sai toàn phần:

$$\text{Var}(R) = \mathbb{E}[\text{Var}(R \mid d)] + \text{Var}(\mathbb{E}[R \mid d]) = 88.125 + 400 = 488.125.$$

Standard deviation một bước khoảng 22.094, dù policy hoàn toàn xác định. Với never-discount đối đầu rule và reset như đề, nhu cầu và sales qua các bước độc lập nên SD của average 20 bước xấp xỉ $\sqrt{488.125/20} = 4.94$. Các SD episode đo quanh 5 là phù hợp với cấu trúc này. Một reward curve gồ ghề vì thế không tự chứng minh policy đang thay đổi.

Khi policy phụ thuộc lịch sử, reward qua bước có thể tương quan; không còn được chia variance cho T một cách máy móc. Dùng sample SD trên run means và protocol nhiều seed giúp tránh một phần sai lầm, nhưng muốn ước lượng uncertainty chính xác cần thiết kế phù hợp với cấu trúc phụ thuộc.

4.14 14. Confidence interval và đơn vị mẫu

Báo cáo có năm training seeds. Mỗi run tạo một policy rồi đánh giá trên một tape riêng. Lấy năm mean để tính SD phản ánh kết hợp biến động học và evaluation. Nếu coi 50.000 bước evaluation gộp là 50.000 mẫu độc lập của chất lượng thuật toán, độ chắc chắn sẽ bị phóng đại vì nhiều bước dùng cùng một policy được học từ cùng run.

Với năm mean, khoảng t minh họa dùng 4 bậc tự do. Khoảng rộng là tín hiệu dữ liệu còn ít hoặc policy không ổn định. Không có ý nghĩa khi giảm interval bằng cách chỉ lặp đánh giá một policy tốt đã chọn rồi bỏ các training seeds kém. Đó là selection bias, không phải tăng độ tin cậy.

So sánh paired nên dùng chênh lệch hai policy trên cùng tape. Tuy nhiên, nếu một số seed cho policy giống hệt baseline, các chênh lệch bằng 0 là cấu trúc có ý nghĩa. Năm seed quá ít để mạnh tay khẳng định thứ hạng phổ quát; bảng từng seed thường cho nhiều thông tin hơn một dấu sao significance.

4.15 15. Robustness: zero-shot shift khác retraining

Trong stress test discount 40%, frozen policy giữ nguyên Q học ở 20% và chỉ thay reward environment. Đây là đo zero-shot robustness trước một thay đổi margin. Retrained policy được học lại ở 40%, đo khả năng tìm policy khi dữ liệu phù hợp chế độ mới. Hai thiết lập trả lời hai câu hỏi khác nhau và không được gộp thành một hàng "agent thích nghi".

Những policy luôn không giảm không bị ảnh hưởng vì action 0 không dùng discount fraction. Những policy còn giảm chịu margin từ 3 xuống 1, nên profit kém hơn. Retraining có thể sửa policy nhưng không bảo đảm từng seed thành công do baseline vẫn có thiếu trạng thái và target noise. Tính học không tạo bảo đảm thích nghi cho một lần train hữu hạn.

Một nghiên cứu robustness đầy đủ nên có lưới demand transition, conversion uplift, cost, opponent và horizon. Chọn tham số trên train/validation regimes, rồi giữ test regimes độc lập. Đo mean, worst-case và regret so với oracle từng regime, thay vì chỉ báo một điều kiện có lợi cho phương pháp đề xuất.

4.16 16. Đối thủ cố định, self-play và cân bằng

Best response với đối thủ cố định tối ưu lợi ích khi policy bên kia không đổi. Self-play là quá trình cả hai cập nhật; evaluation cuối chỉ quan sát chất lượng cặp policy đã đóng băng.

Nash equilibrium yêu cầu không bên nào có thể tăng lợi ích bằng cách đơn phương đổi policy trong lớp trò chơi đã định nghĩa. Một trận có reward gần nhau không chứng minh điều kiện này.

Trong lab, cấu trúc reward khiến không giảm là rất mạnh trước rule, nhưng không nên tự tuyên bố mọi policy self-play đã hội tụ cân bằng. Cần tính hoặc xấp xỉ best response với từng opponent snapshot và đo exploitability. Với không gian nhỏ, có thể liệt kê 16 policy reactive theo bốn observation; với policy có lịch sử đầy đủ, lớp phương án rộng hơn nhiều.

Cross-play giữa checkpoint và seed giúp phát hiện co-adaptation: hai policy tốt khi chơi cùng nhau nhưng kém với đối thủ khác. Bộ bài có self-play và dữ liệu cuối trên năm seed, chưa triển khai toàn bộ ma trận cross-play. Vì vậy báo cáo giữ kết luận ở mức minh họa non-stationarity và biến động kết quả, không gán chứng nhận equilibrium.

4.17 17. Từ thuật toán sang thiết kế có thể tái lập

Một kết quả tái lập cần nguồn, tham số, seed, phiên bản thư viện, protocol và dữ liệu đầu ra. Chỉ công bố seed 42 thiếu thông tin về số lần gọi RNG, cách phá hòa, cách reset và việc evaluation có cập nhật Q không. Hai chương trình cùng dùng seed 42 nhưng tách stream khác nhau có thể sinh các policy rất khác mà vẫn cùng mô hình phân phối.

Bộ bài lưu bản gốc, bản mở rộng, Q snapshot, từng episode, summary và version. Notebook đọc dữ liệu đầy đủ đã lưu và chạy thêm các ví dụ kiểm chứng nhỏ; script `--episodes 5000` tái tạo thí nghiệm chính. Sự khác biệt giữa "đọc kết quả đã chạy" và "rerun toàn bộ training" được ghi ở notebook và README.

Nếu môi trường phần mềm thay đổi, bit-level reproducibility của floating point hoặc backend có thể khác. Vì bài chủ yếu dùng NumPy và mạng CPU nhỏ, kiểm tra nên ưu tiên tái lập hành vi và protocol, đồng thời lưu version thực tế. Không được âm thầm thay kết quả cũ rồi giữ cùng mô tả phiên bản.

4.18 18. Phạm vi đóng góp của bộ báo cáo

Bộ bài cung cấp một triển khai học tập có nguồn và bằng chứng kiểm. Phân tích MAS chuyển taxonomy thành hợp đồng và failure injection; phân tích RL chuyển MDP thành phép kiểm trạng thái, oracle và exact policy evaluation. Những phần này làm rõ hai paper bằng một hệ có thể thực hiện, thay vì chỉ tóm tắt abstract.

Các giới hạn vẫn rõ: không gọi LLM, không tái lập các benchmark của AutoGen hoặc MetaGPT, không có dữ liệu intent thật và không có thị trường kinh tế đã hiệu chỉnh. Các đề xuất mở rộng được viết như hướng nghiên cứu để tiếp tục, có protocol và tiêu chí đo, không có kết quả được bịa cho những thí nghiệm chưa thực hiện.

5 Tổng hợp, phản biện và thiết kế nối hai hướng nghiên cứu

5.1 Bảng đối chiếu mục tiêu và loại bằng chứng

Khía cạnh	Survey MAS/LLM	Survey RL
Câu hỏi trung tâm	Ai phối hợp, bằng kênh nào, theo cơ chế nào?	Policy học từ trải nghiệm như thế nào?
Đơn vị phân tích	Tác tử, kênh, tổ chức, kiến trúc	State, action, reward, return, value
Vai trò dữ liệu	Ngữ cảnh, bằng chứng, message, mô hình chia sẻ	Transition và feedback từ tương tác
Rủi ro	Lỗi lan truyền, đồng thuận sai, overhead	Reward lệch, mẫu thiếu, state aliasing, bất ổn học
Đánh giá cần có	Quality, cost, latency, trace, ablation	Frozen evaluation, seed, oracle, robustness
Liên hệ bộ mã	Shopping MAS và failure injection	E-commerce Q-learning và DP

5.2 Hai cấp độ thích nghi không nên gộp

Điều phối động có thể thay graph thực hiện mà không cập nhật trọng số một mô hình nào. Một LLM đọc feedback và sửa kế hoạch trong cùng context có thể thay hành vi, nhưng đó chưa chứng minh có thuật toán RL tối ưu tham số. Ngược lại, Q-learning có thể cập nhật policy trong một môi trường có giao thức hoàn toàn cố định. "Agent biết thích nghi" vì thế cần chỉ rõ thứ gì thay đổi và theo cơ chế nào.

Trong tutorial, classifier được học offline bằng nhãn và giữ cố định trong inference. Coordinator thay trạng thái theo kết quả công cụ nhưng không học policy phân việc. Trong assignment, Q được cập nhật trong training rồi đóng băng để đánh giá. Ba loại thay đổi này cùng xuất hiện trong hệ thông minh nhưng có mục tiêu và bằng chứng khác nhau.

5.3 Thiết kế tham chiếu: trợ lý kinh doanh có tầng quyết định và tầng kiểm chứng

Một kiến trúc nối hai hướng có thể gồm tầng nhận yêu cầu, tầng dự đoán, tầng chọn hành động và tầng kiểm chứng. Intent classifier định tuyến nhiệm vụ; agent dữ liệu thu thập trạng thái có version; policy learner đề xuất mức khuyến mãi; policy validator kiểm ràng buộc; coordinator quản lý trạng thái và dịch vụ thực thi trả xác nhận. Proposal không được coi là một giao dịch đã thành công.

State cho learner nên chứa nhu cầu, tồn kho, lịch sử giá, ngân sách khuyến mãi và thông tin đối thủ được phép quan sát. Action là các mức giảm hợp lệ hoặc đề xuất giữ giá. Reward có thể dựa trên profit đã trừ chi phí liên quan, nhưng các điều kiện pháp lý/nghiệp vụ bắt buộc phải nằm ngoài score học. Nếu phản hồi đến muộn, cơ chế gán reward cần được thiết kế trước khi train.

Đây là đề xuất kiến trúc, chưa được triển khai thành dịch vụ kinh doanh. Bộ bài thực thi hai phần nhỏ có thể kiểm chứng: coordinator shopping và learner bán hàng toy. Chúng chưa tích hợp thành hệ tự đổi giá thật; vì thế báo cáo không đưa con số doanh thu hoặc lợi ích vận hành cho kiến trúc mở rộng.

5.4 Thiết kế thí nghiệm để tách lợi ích chia vai khỏi tăng tính toán

So sánh ít nhất bốn điều kiện: một agent với ngân sách chuẩn; một agent với cùng tổng ngân sách như MAS; MAS đầy đủ; MAS bỏ một cơ chế phối hợp quan trọng. Mọi điều kiện dùng cùng dữ liệu, quyền công cụ, tập tác vụ và tiêu chí thành công. Nếu MAS được thêm nguồn dữ liệu, cần có baseline cùng nguồn để tách ảnh hưởng dữ liệu khỏi tổ chức.

Đo success, groundedness, constraint violation, cost và latency; báo cáo cả phân phối và các trường hợp thất bại. Với LLM, lưu prompt/model version, nhiệt độ, số lần lấy mẫu, token và tool traces. Với RL, lưu training steps, evaluation tapes, seed và snapshot policy. Chỉ tăng số lần chạy mà không giữ protocol không tạo bằng chứng tái lập tốt.

Một ablation có ý nghĩa là bỏ bước inventory verification trong khi giữ số agent khác. Nếu chất lượng giảm, kết luận hẹp là verification có ích trong tập ca, chưa phải mọi cách chia vai đều có ích. Đây chính là cách đọc thí nghiệm B3 của tutorial. Nhận diện đúng biến độc lập giúp tránh gán công lao cho một nhãn kiến trúc rộng.

5.5 Đề xuất kiểm tra độ bền nhiều tầng

Tầng dữ liệu: thêm thiếu trường, schema sai, dữ liệu cũ và nguồn mâu thuẫn. Tầng giao tiếp: thêm timeout, duplicate response, sai task ID và response đến muộn. Tầng quyết định: đổi giá vốn, discount, đối thủ và nhu cầu. Tầng triển khai: đo tải, version và cơ chế khôi phục. Mỗi thay đổi cần giữ một tập đối chứng để xác định tác động.

Bộ mã hiện kiểm schema, freshness, timeout, forged output và shift discount; đồng thời có self-play để minh họa đồng học. Nó chưa kiểm tải mạng, Byzantine agent, đối thủ ngoài tập hoặc deployment thật. Một bản báo cáo chi tiết phải ghi cả những gì đã đo và những gì chưa đo để người đọc đặt đúng độ tin cậy.

5.6 Những luận điểm cần tránh sau khi đọc hai paper

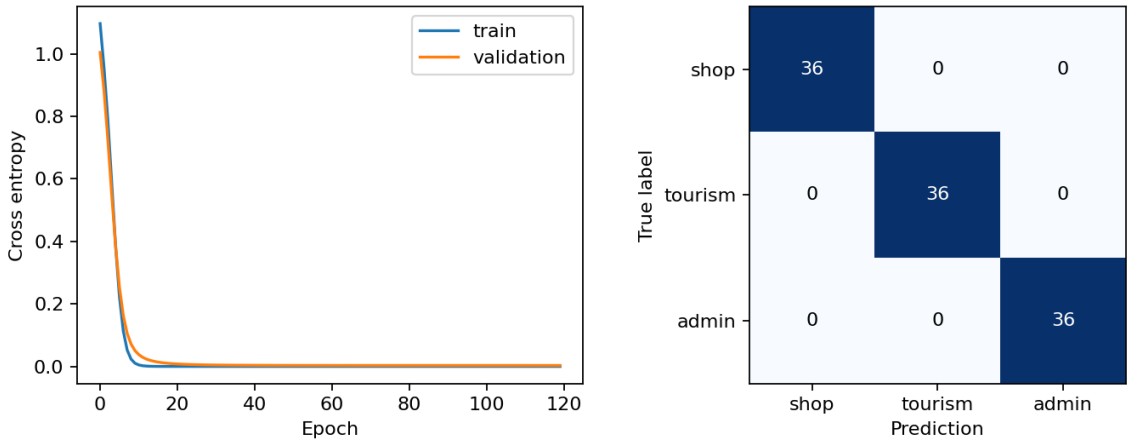
Không suy ra "nhiều agent hơn luôn đúng hơn"; sai số tương quan và chi phí giao tiếp có thể làm kết quả kém đi. Không suy ra "RL luôn học policy tối ưu"; giả định Markov, tính dừng, độ bao phủ và protocol đóng vai trò thiết yếu. Không suy ra "accuracy cao là hệ đáng tin"; một classifier đúng trên synthetic test vẫn nhận sai OOD. Không suy ra "rule-based không thích ứng đầu vào"; nó có thể phản ứng theo state dù không cập nhật từ reward.

Các kết luận tích cực vẫn rất rõ: chia vai có thể tạo hợp đồng kiểm được; RL cung cấp cơ chế cải thiện từ feedback; model-based oracle giúp đo khoảng cách tới chuẩn trong môi trường biết mô hình; và logging/ablation giúp truy nguyên lỗi. Giá trị của hai paper nằm ở cách đặt vấn đề, cộng với kiểm chứng cụ thể khi áp dụng.

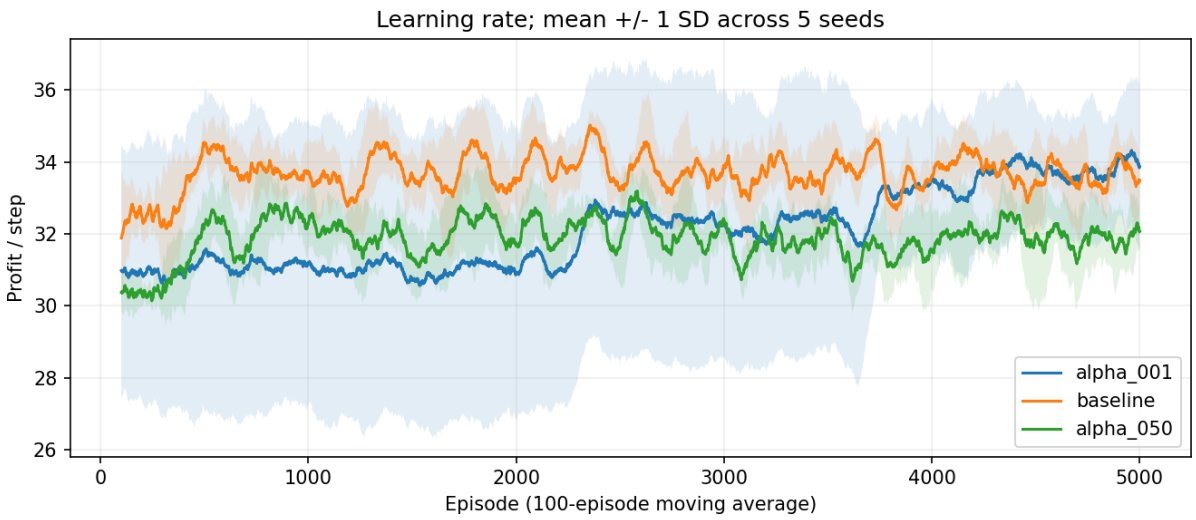
6 Bằng chứng minh họa liên hệ hai paper

Các kết quả dưới thuộc bộ mã của báo cáo, không phải bảng benchmark chép từ hai bài gốc. Tutorial kiểm kiến trúc local; assignment kiểm policy trong mô phỏng. Phần số liệu đầy đủ nằm trong hai báo cáo chuyên biệt và các CSV đi kèm.

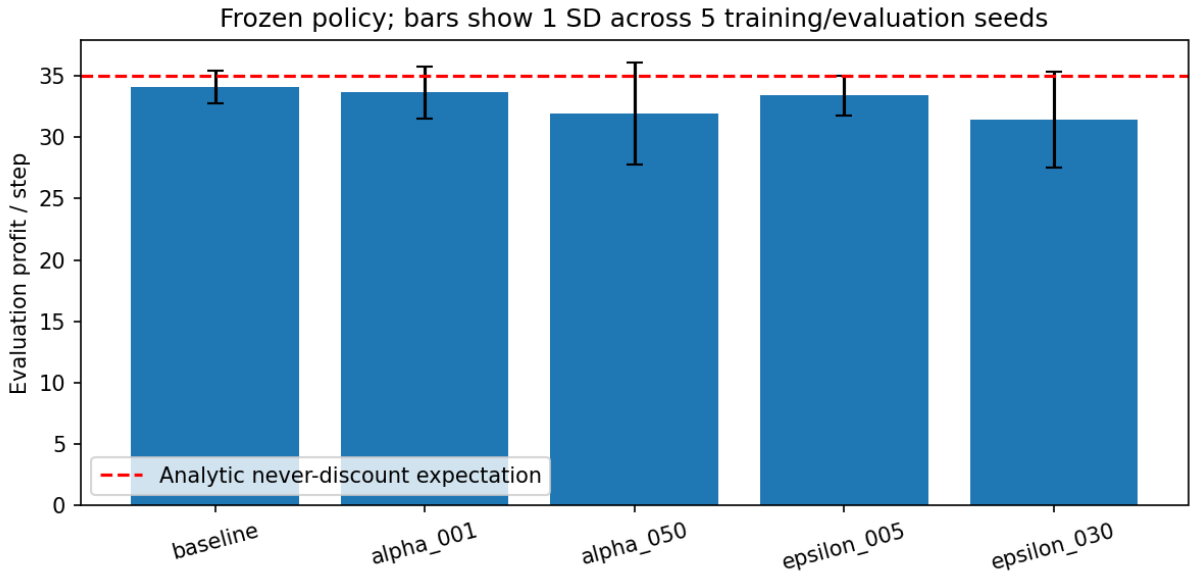
Thực nghiệm của báo cáo	Quan sát	Phạm vi kết luận
MAS B0/B1	Cùng 100% success trên 11 ca	Chia vai chưa tăng chất lượng trong catalog nhỏ
MAS B3	8/11 ca đúng; lỗi khi bỏ kiểm	Validation cần thiết trong tập failure injection
Intent test	108/108 đúng	Chỉ trên test tổng hợp chia nhóm
Intent OOD	4/6 clarify; 2 false accept	Softmax/confidence chưa đủ phát hiện ngoài miền
RL 5 seeds	34.0997 ± 1.3678	Baseline còn biến động và dưới policy không giảm
Rule/DP	Cùng policy và khoảng 35.1295	Best response trong mô hình đối thủ đã chỉ định



Hình 1: Minh họa phân biệt chất lượng trong phân phối và đánh giá ngoài miền; đây là dữ liệu tự thực thi.



Hình 2: Độ nhạy learning rate trong mô hình bán hàng toy; vùng tô là một SD qua năm seed.



Hình 3: Đánh giá policy đóng băng và mốc analytic không giảm giá.

7 Phụ lục đọc paper: bản đồ nội dung và câu hỏi kiểm tra

7.1 Đối chiếu Paper 1 với tài liệu gốc

Phần tài liệu gốc	Nội dung đã phân tích	Câu hỏi khi đọc lại
Section 1	Động lực, đóng góp, vị trí survey	Tại sao cơ chế phối hợp là trọng tâm?
Section 2	MAS, LLM, collaborative AI	Mô hình khác agent ở trách nhiệm nào?
Section 3	Agent tuple, system, collaboration channel	Kênh cần mô tả đủ những thuộc tính gì?

Phần tài liệu gốc	Nội dung đã phân tích	Câu hỏi khi đọc lại
Section 4.2	Cooperation, competition, coopetition	Mục tiêu cấp agent và cấp hệ có khác không?
Section 4.3	Rule-, role-, model-based	Đây là protocol hay decision policy?
Section 4.4	Centralized, decentralized/distributed, hierarchical	Quyền kiểm soát khác vị trí thực thi thế nào?
Section 4.5	Static/dynamic orchestration	Điểm nào thật sự thay đổi lúc chạy?
Section 5	Mạng, QA/NLG, xã hội/văn hóa	Ground truth và authority ở đâu?
Section 6	Collective intelligence, benchmark, safety	Bằng chứng nào còn thiếu?
Section 7	Kết luận taxonomy	Khung có giúp thiết kế phép thử không?

7.2 Đối chiếu Paper 2 với tài liệu gốc

Phần tài liệu gốc	Nội dung đã phân tích	Câu hỏi khi đọc lại
I–II	Mục tiêu, MDP/POMDP, policy, Bellman	State có đủ và quá trình có dừng không?
III	Bandit, exploration, value	Feedback khác nhãn giám sát thế nào?
IV-A	Dynamic programming	Biết mô hình có thể xây oracle nào?
IV-B	Monte Carlo	Return cần chờ đến đâu và nhiễu từ đâu?
IV-C	TD, SARSA, Q-learning	Target theo policy nào và có bootstrap không?
IV-D/E	Actor–critic, Bayesian	Mỗi mô hình ước lượng điều gì?
IV-F	DQN, replay và target network	Vì sao cần ổn định khi xấp xỉ Q?
V-A đến V-I	Các miền ứng dụng	Quyết định có ảnh hưởng tương lai thật không?
V-J/K	Challenges, lessons learned	Reward, coverage, non-stationarity có được kiểm?
VI–VII	Discussion và kết luận	Phát biểu hội tụ cần điều kiện gì?

7.3 Câu hỏi phản biện và lời giải định hướng

Vì sao hai agent đồng ý vẫn có thể sai? Vì cùng mô hình hoặc cùng nguồn có thể gây lỗi tương quan; đa số không tạo nguồn thông tin mới. Cần đối chiếu bằng chứng hoặc phép kiểm độc lập.

Vì sao cùng state–action có reward khác nhau chưa chắc là non-stationarity? Môi trường stochastic vốn tạo reward khác qua các mẫu. Non-stationarity nói quy luật phân phối thay đổi; cần nhiều dữ liệu hoặc kiểm soát policy đối thủ để phân biệt.

Vì sao tăng số seed chưa chứng minh mô hình kinh tế đúng? Seed chỉ thay chuỗi ngẫu nhiên trong mô hình đã giả định. Nếu conversion sai hoặc thiếu thị phần, mọi seed đều chịu cùng sai lệch cấu trúc.

Vì sao một agent biết mô hình có thể là baseline tốt? Nó giúp xác định khả năng đạt được trong mô phỏng nhỏ; chênh lệch với learner cho thấy khoảng cách cần cải thiện. Nhưng lợi thế biết mô hình phải được nêu để không làm so sánh thiếu công bằng về thông tin.

Vì sao không dùng DQN cho bảng tám ô? Không gian nhỏ không đòi xấp xỉ bằng mạng; mạng thêm nhiễu và tuning. Chọn tabular giúp minh bạch cập nhật và dễ đối chiếu oracle.

Vì sao ngưỡng confidence cần validation? Chọn trên test làm rò rỉ thông tin về benchmark cuối. Dù chọn đúng validation, confidence vẫn có thể cao với OOD nên cần đánh giá false acceptance riêng.

Khi nào nên dùng MAS? Khi có trách nhiệm, dữ liệu hoặc quyền tách biệt và phối hợp tạo lợi ích đo được. Nếu baseline đơn đạt cùng chất lượng với ít chi phí, lựa chọn đơn giản vẫn có cơ sở.

Khi nào nên dùng RL? Khi có chuỗi quyết định, feedback và điều kiện tương tác phù hợp để học. Nếu biết mô hình nhỏ và có lời giải chính xác, planning hoặc luật đã được kiểm chứng có thể phù hợp hơn.

8 Kết luận chung của báo cáo paper

Hai paper cung cấp hai mặt bổ sung của hệ tác tử: tổ chức tương tác và học quyết định. Một thiết kế có trách nhiệm cần cả mô hình khái niệm rõ lẫn bằng chứng thực nghiệm phù hợp. Bản báo cáo đã trình bày nền tảng, phân loại, công thức, thuật toán, ứng dụng, phản biện và liên hệ trực tiếp với mã đã chạy.

Giới hạn xuyên suốt là mô phỏng toy và dữ liệu intent tổng hợp. Các con số thực thi chứng minh quy trình và kiểm tra một số giả thuyết hẹp, không thay thế benchmark LLM hay đánh giá vận hành thực tế. Đó là ranh giới giúp sử dụng kiến thức từ survey một cách chính xác và có thể kiểm tra.

Tài liệu tham khảo

[1] PGS.TS. Trần Đình Quế (tài liệu môn học được cung cấp), *Developing Multiagent Systems: A Practical Tutorial with Python and PyTorch*, slide_multiAgent_system_20.10.2026.pdf, 25 trang, 2026. Nguồn cục bộ trong đề bài.

[2] *Laboratory Experiment 02: Reinforcement-Learning-Based Agents versus Rule-Based Agents*, Intelligent System Development, 2026–2027, assignment_Agent02_due_29.10.pdf, 13 trang; agent02_code.tex và agent02_complementary_code.pdf. Tài liệu cục bộ do người dùng cung cấp.

[3] K.-T. Tran, D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O'Sullivan, H. D. Nguyen, *Multi-Agent Collaboration Mechanisms: A Survey of LLMs*, arXiv:2501.06322v1, 10/01/2025. https://arxiv.org/abs/2501.06322v1. Phân tích trực tiếp bản PDF 35 trang được cung cấp.

[4] M. Naeem, S. T. H. Rizvi, A. Coronato, *A Gentle Introduction to Reinforcement Learning and Its Application in Different Fields*, IEEE Access, vol. 8, pp. 209320–209344, 2020. DOI: 10.1109/ACCESS.2020.3038605. Phân tích trực tiếp bản PDF 25 trang được cung cấp.

[5] R. S. Sutton, A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018. Tài liệu nền tảng được cả slide và paper RL dẫn chiếu.

[6] C. J. C. H. Watkins, P. Dayan, *Q-learning*, Machine Learning 8, 279–292, 1992. DOI: 10.1007/BF00992698.

[7] Q. Wu et al., *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*, 2023. https://arxiv.org/abs/2308.08155.

[8] S. Hong et al., *MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework*, 2023, bản cập nhật 2024. https://arxiv.org/abs/2308.00352.

[9] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, I. Mordatch, *Improving Factuality and Reasoning in Language Models through Multiagent Debate*, 2023. https://arxiv.org/abs/2305.14325.

[10] PyTorch, *CrossEntropyLoss*, tài liệu chính thức. https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html. Đối chiếu giao diện ngày 28/10/2026; môi trường thực nghiệm dùng PyTorch 2.8.0.

[11] Python documentation, *Coroutines and Tasks: asyncio.wait_for*. https://docs.python.org/3/library/asyncio-task.html. Phần timeout được kiểm trực tiếp bằng tests trong môi trường thực thi.

[12] Bộ mã và dữ liệu tự thực thi của báo cáo: tutorial/, assignment/, results/, build/; seed và cấu hình lưu trong summary.json, intent_metrics.json và mas_metrics.json, ngày 28/10/2026. Đây là nguồn kết quả mô phỏng, không phải nguồn dữ liệu thị trường thực tế.